POT JS

MANUAL

POT JS is a framework for Javascript that allows to effortlessly store, update and retrieve values to a distributed, always-on storage network.

It is the Javascript API for the Proximity Order Trie (POT) implementation in Go that allows to write and read key-value pairs, to and from the decentralized, immutable storage network Swarm, the web3 storage layer. With a custom persister, any other storage could be used. An in-memory storage mode is included in POT and POT JS.

The POT storage tree structure allows to have updateable key-value maps stored on an otherwise immutable, decentralized, content-addressable network. It respects immutability and translates the flat key space to an updateable trie that describes membership. It is called Proximity Order Trie after a new storage technique that is described in a forthcoming paper.

This manual covers the use of POT JS, and offers hints and caveats for maintainers and potential contributors to the project.

INDEX

INDEX 2

USE 4

QUICK START 4
Example 4

INTRODUCTION 5
Brief 5
Proximity Order Trie 6

CODING 7
Initialization 7
Storage 8
Persisting 8
The Save Reference 9
Key and Values: Types and Sizes 9
Network Connection 11
Timeout 12
Cancel 13
Batch ID 15
Installing A Local Swarm network 16
Explicit Initialization 17
Synchronous Initialization 17
Verbosity 18
Error Propagation 20
Crashes 20
Further Reading & Support 21

TUTORIAL 22
Installation 22
Tutorial #1 — Hello, P.O.T.! 23
Tutorial #2 — Moving to the Browser 26
Tutorial #3 — Sub-Component Integrity 27
Tutorial #4 — Save, Load, and Delete 28
Tutorial #5 & 6 — Interaction 30
Avoiding Deadlocks 36
Tutorial #8 — Verbosity 37
Tutorial #11 — Node App 40
Tutorial #12 — Persistence 43

FUNCTION OVERVIEW 47

EXAMPLES 49
Example 1: Minimal 50
Example 2: Interaction 51
Example 3: Root, Delete, alternate Initialization 54
Example 4: Synchronous 58
Example 5: Network 59
Example 6: Network II 61
Example 7: Node 64

Example 8: Node Web App *65*
Example 9: Node App / Network III *69*
Example 10: Cancelation *71*
Example 11: Error Handling *73*

BUILDING **76**

Clean Build *77*
Requirements *78*
POT JS *78*
GO POT *78*

TESTS 79

Installation Tests *79*
Deep Tests *81*
Running a Deep Test *84*

DEVELOPING **88**

FILES 89

MAKE 90

DEVELOPER NOTES 92

A JS API implemented in Go *92*
Notes on Debugging *96*
Note on syscall/js *96*
Javascript Rewrite *97*
Initialization *98*
Types *99*
Keys *101*
Tests *102*
Error Handling *104*
Slots *105*
Node.js Swarm Persister *109*
Synchronous Architecture *110*
Garbage Collection *112*

ROADMAP **115**

USE

QUICK START

- Clone **http://github.com/brainiac-five/potjs**
- Drop **lib/pot.wasm**, and **pot-web.js** or **pot-node.js** in a directory.
- Write some Javascript using **new**, and functions **put()**, **get()**, **delete()**, **save()**, and **load()**.
- Check out the functions overview (pg. 47), reference (separate doc) and examples (pg. 48)

There is no need to *build* the project, i.e., no need to install or run Go, to *use* it. Only **pot.wasm** and either **pot-web.js** or **pot-node.js** are needed, as they come. They are in the **lib/** folder.

EXAMPLE

```
<script src="./lib/pot-web.js"></script>

<script>

(async () => {

    await pot.ready()
    kvs = new pot.Kvs()
    await kvs.put("hello", "POT!")
    value = await kvs.get("hello")
    console.log("key ‹hello› has:", value)
})()

</script>
```

To run this example:

```
cd potjs/tutorial
npx http-server -c1 . &
open http://localhost:8080/t0.html
```

Check the browser console.

tutorial/t0.html

Node.js works analogous, see **examples/example7.js**, pg. 64

INTRODUCTION

This section is an introduction to how POT JS is used. The basics are simple and fit on this one page.

After this brief, you may choose to work through the structured explanation that follows to get a deeper understanding; or skip to the hands-on tutorial (pg. 22); or you might decide to focus on the function reference (separate doc) to beat your own path. The final chapters of this manual (from pg. 71) explain finer points of how Go and Javascript work together to support contributors and maintainers.

BRIEF

POT JS provides a simple way to store and retrieve *mutable* key-value pairs on the *immutable*, decentralized storage network Swarm.[1] To use POT JS in the browser, include:

```
<script src="pot-web.js"></script>
```

For node.js, require:

```
require("pot-node")
```

The main primitives are:

```
kvs = new pot.Kvs(swarm, batch)
await kvs.put("some key", "some value")
value = await kvs.get("some key")
await kvs.delete("some key")
```

This is almost a complete example program (cf. example previous page).

Data is always put right to storage. To retrieve the root of a KVS, to be able to retrieve the entire KVS at a later point (instead of using `new` to create a new one), use:

```
ref = await kvs.save()
kvs = await pot.load(ref)
```

There are functions for raw byte access, for synchronous access, and for tests. See the function overview, pg. 47.

[1] https://ethswarm.org/

PROXIMITY ORDER TRIE

POT JS makes the functionality of a proximity order trie (POT) available in the browser or with node.js. This page gives a high-level description of what POT is for and how it works.

A proximity order tree is a data structure that allows to operate a key-value store (KVS) on an immutable, content-addressed[2] storage network. It makes it possible to effortlessly use Swarm as a key-value database – to store a map-like structure to it – including being able to change values.

This sounds like an oxymoron: an immutable storage would seem to not allow for any update of values. Additionally, Swarm is content-addressed: the address of any content is always the hash of the content,[3] not an arbitrary key.

POT solves this by exploiting a feature observed in a Merkle trie when its nodes are hashed: after an update to the trie, there is one path back from the new or changed node to the root that has to be re-hashed every step of the way to arrive at the new root hash. All other hashes in the trie remain the same, as their pre-images have not changed. But the root hash always changes. That's the point with Merkle tries. POT uses this attribute to implement an *update* to data stored in the proximity order trie as a *change* of the root hash.[4] In essence, with every update, you get a different root hash back that is a handle specifically for the current state of the KVS, i.e. the current collection of key-value pairs. The keys and values are both data from the point of view of Swarm. They are inter-connected and grouped in an efficient way – the proximity order trie – and stored in 'chunks,' Swarm's uniform data storage unit.

Because a network like Swarm functionally provides the inverse to the hashing process – a lookup returns the pre-image to a hash, by the definition of what content-addressing is – the update process of a Merkle trie can be 'reversed' using Swarm and repurposed to 'emulate' a data update on an immutable storage: a new root hash is be created on every update, and serve as the new entry point to the now-valid group of chunks that contain the KVS data. Deleted or changed nodes fall out of the trie, even though they are still stored in the immutable storage. Updated key-value pairs are added into the trie. All entries, old and new, still exist in-storage. But some are not part of the hash tree anymore that represents the current KVS state.

The keys of the KVS, are not the native Swarm addresses, which retrieve pre-images of hashes. To search and retrieve a value for a POT key is therefore not something Swarm's base layer can do. POT uses intricate bit patterns to implement the performant navigation of this higher data layer. In fact, POT *is* the complex algorithm that makes the system 'understand' where in the trie to find the value to a key – or where to insert a new key-value pair: the bit patterns of a key's hash is used to establish a logarithmic-time search order. This is described in a forthcoming paper.

POT JS uses the implementation of POT in Go that performs the actual number crunching for this:

`https://github.com/ethersphere/proximity-order-trie`

---

[2] Content-addressed means that the storage address is produced from the value that is stored. If the value changes, the address changes, too. A value can therefore not be updated without losing its address.

[3] In some cases, the hash is of the content plus additional ID data.

[4] The values of this KVS are not the values stored in Swarm. They have keys that are independent of their values. Key and value are stored together and constitute the value of the Swarm record.

CODING

POT JS is for storing key-value sets on immutable, distributed storage networks – specifically on Swarm. It works in the browser and with node.js.

This section (pp. 7 to 22) goes through the basics and includes advanced topics and caveats. Another way to get started are the **tutorial** (pg. 22), the **examples** (pg. 48), the **function overview** (pg. 47) and the **function reference** (separate document).

INITIALIZATION

BROWSER

For use in the browser, include **pot-web.js**, then wait for the initialization to finish (**pot.ready()**):

```
<script src="pot-web.js"></script>
<script>
(async () => {
    await pot.ready()
    ...
```

examples/example1.html

In case **pot.wasm**, the WASM executable, is not in the same folder as **pot-web.js**, a path to **pot.wasm** can be provided using the **wasm** attribute of the script tag.

```
<script src="pot-web.js" wasm="lib/pot.wasm"></script>
```

cf. examples/example2.html

NODE.JS

For use with node.js, include **pot-node.js**, then wait for the initialization to finish:

```
require("pot-node")
; (async () => {
    await pot.ready()
   ...
```

examples/example7.js

If a path to **pot.wasm** must be provided, add an additional parameter to **require** like this:

```
require("pot-node")("lib/pot.wasm")
```

Note, this throws an error if there *is* a **pot.wasm** file in the same directory as **pot-node.js**.

For an alternate way of initializing see example 3 (pg. 54) and the notes on initialization (pg. 98).

STORAGE

After `pot.ready()` resolves, the JS `pot` object can be used to create a new key-value store (KVS). Use its `put()` and `get()` methods to store and retrieve key-value pairs to and from it.

```
kvs = new pot.Kvs()
await kvs.put("hello", "Hello, P.O.T.!")
value = await kvs.get("hello")
```

examples/example1.html

Key-value pairs are deleted from the KVS with `delete()`.

```
await kvs.delete("hello")
```

cf. examples/example3.html

Trying to retrieve a deleted key's value returns `undefined`, the same as for keys that were never assigned a value. `null` is a valid value that can be stored and retrieved with `put` and `get`.

If `new` calls the KVS constructor without arguments (like `new pot.Kvs()` in the example above) then **in-memory storage** is used for the KVS. This is useful for testing, prototyping, and some special applications. In this mode, values are not permanently stored to a network and are lost when the program is stopped. And what was 'saved' with `store()` cannot be retrieved when the program is restarted. `new pot.Kvs(null, null)` likewise triggers the use of in-memory storage.

To use a Swarm **network** as storage, `pot.Kvs()` needs two parameters: the URL of the API of a Bee client of the network, and a postage batch id that pays for the storage. See pg. 11 for details.

See `examples/example5.html` (pg. 59) for a minimal, complete program that use a network.

PERSISTING

Every `put()` is immediately persisted. To be able to retrieve a KVS later, use `save()`. This returns a save reference. To retrieve a KVS at a later point, use `load()` with that save reference and the same URL and batch ID as the constructor call.

```
ref = await kvs.save()

kvs2 = await pot.load(ref, beeUrl, batchId)
```

Note that `save()` does not persist *in-memory* storage across program starts (see previous section). In tests,[5] you will often see `pot.load(ref)` calls with only one parameter. This is of limited use otherwise. `save()` and `load()` only really make sense when using a network as storage. But they are available when using in-memory storage to facilitate learning, development, and testing.

THE SAVE REFERENCE

The reference returned from `kvs.save()` is technically the root of a POT trie (cf. pg. 6). Relevantly, it identifies one specific *state* of the set of key-values that is a KVS. It is *not* tied to the *identity* of a specific KVS object, but, like a hash belongs to its pre-image, the save reference represents a specific state of a KVS. The reference is accordingly also not tied to a specific variable that contains a KVS. And the reference is not tracking the changes to a KVS after the `save()` call that the reference was obtained from. It will at all times through `load()` return the same KVS *state,* i.e., the exact same set of key-value pairs the KVS consisted of when saved. (That is, until the postage batch (see below) that was used expires and the data may be purged from Swarm; or until the Swarm test network used for storing the KVS is shut down.)

A different reference will be returned upon saving the *same* KVS object again when it was updated in the meantime (values added, deleted or changed). Also, the same reference will retrieve the same KVS state regardless of whether the KVS that was originally saved is later updated, saved again or not, or if a KVS that was loaded through this reference gets updated, and saved again or not. Any updated KVS will have a different reference returned from `save()` – unless the updates in their sum result in the same state as when it was saved. Creating the exact same KVS from scratch by the same sequence of `put()` calls will result in the same reference when saving.

All this is true for in-memory storage, the same as when storing to a network.

KEY AND VALUES: TYPES AND SIZES

A key can be a `string`, a `number`, or a `Uint8Array` of bytes. It can be up to 32 bytes.

A value can be a `string`, a `number`, a `Uint8Array` of bytes, a `Boolean`, or `null`.

The value size limit depends on the system set up. It is currently markedly smaller for node.js than for in-browser use: It can be around 100,000 bytes for node.js, but ten million bytes in the browser. These limitations, especially for node.js, are likely to be improved in the future.

A **soft cap** for value sizes exists that defaults to 100,000 bytes. Trying to store a value bigger than this size will result in an error. The cap can be set to an arbitrary value using `setValueLimit()`. Changing the value does not per se change what the underlying system will be able to absorb, it just helps to protect an application.

[5] `test/suites.js`, `test/jest.async.js` etc.

VALUE TYPES

When using `put()` and `get()` (or `putSync()` or `getSync()`), a value that was stored will be returned with the same type that it was stored. E.g., a number will be returned as a Javascript number, not as a string. On the binary level, type information is stored with the data, as part of the value.

But for values stored with `putRaw()` or `putRawSync()`, no type information is stored with the data. To retrieve values stored with `putRaw()` or `putRawSync()`, *raw access* functions exist whose names indicate the desired type of a returned value. They are: `getBoolean()`, `getNumber()`, `getString()`, `getBooleanSync()`, `getNumberSync()`, and `getStringSync()`. Their function names indicate as what type a raw binary value will be interpreted. They are strictly for reading data stored with `putRaw()`. They are not for casting types. E.g., a string **"1"** stored with `putRaw()` will *not* return a `1` when read with `getNumber()` but throw an error as internally it would be missing seven more bytes. **"1"** stored with `putRaw()` must be read with `getString()`.

Raw values give more control to a user to store more complex data structures. But they are incompatible with the normal `put()` and `get()` (and `putSync()` and `getSync()`) methods. The reason is that data stored with `put()` has a type byte prepended that will show up when reading it back with `getRaw()`, and will be misinterpreted by any of the `getBoolean()`, `getNumber()` etc. methods. Data stored with `putRaw()` does not have that leading type byte. Accordingly, reading it back with `get()` will (most of the time) lead to a wrongly interpreted value or to errors.

These two competing approaches overlap with a third one: storing and retrieving `Uint8Array`s, using only `putRaw()`, and `getRaw()`. This leaves the task to cast into the right value to the retrieving code, and presents the highest degree of freedom to store custom data structures.

KEY AMBIGUITY

Keys can be Javascript `numbers`, up to 32 byte long `strings`, or `Uint8Arrays`.

Keys are not bijective: A `number`, a `string` and a `Uint8Array` may end up being the same binary byte sequence and thus the same key. There are multiple ways to represent the same byte sequence that a key consists of and there is no type-coding for keys as there is for values.

All keys are internally padded with zeros for a size of 32 bytes. Note that Unicode characters may use more than one byte per character, `s.length` returns the number of bytes used in a string `s`.[6]

Javascript does not differentiate between integer and floating points numbers, all numbers are 64 bit IEEE 754 floats internally. Used as key, numbers are converted to their float byte representation; a key becomes an 8-byte value, followed by 24 zero bytes. The IEEE 754 bytes are stored instead of going by the digits of a number because there is no strong way in Javascript to tell integers and floats apart. There are no `BigInt` keys as POT JS[7] cannot handle Javascript `BigInt`s. Thus, `"1"`, `1`,

[6] https://developer.mozilla.org/de/docs/Web/JavaScript/Reference/Global_Objects/String/length

[7] Due to a limitation of the Go package that interfaces with Javascript, syscall/js.

and "0x01" are each different keys but 1, 1.0, 0x1 and 01 are the same. Precision can be lost for decimal fractions or for very high numbers – e.g., when using literals – the same way it can always be lost for Javascript's floats. The diverse NaN error conditions[8] are effectively different keys. This can help retracing errors where NaN was used as key inadvertently.

Strings and byte arrays are length-checked to be <= 32 byte and cause an error if they are longer.

The byte arrays [0], [0,0] etc. are the same key, as is [1], [1,0], or [1, 0, 0] because of the internal zero padding. But most relevantly, the byte array [65] is the same key as the number 131,072 (because 0x4100000000000000 represents 131,072 in 64 bit IEEE 754) , "A", or "A\0" (because 0x41 is UTF-16 for "A"), and so is [65, 0, 0, 0, 0, 0]. They are all converted to the binary key

0x4100000000000000000000000000000000000000000000000000000000000000.

This ambiguity is tolerated to allow for full control over the actual binary key value.

NETWORK CONNECTION

To connect with a Swarm network, the constructor call new pot.Kvs() requires two parameters:

1. a URL to a Swarm bee node http API and
2. a Swarm batch ID to pay for storage.[9]

For example (cf. examples/example5.html and pg. 59), the arguments shown below connect the KVS that is created with a local Swarm test network:

```
map = new pot.Kvs("http://localhost:1633",
        "ab642e87327ec31816ed7aa675b4e92ea2c3d92c5022a599604c8bd8ddfe545d")
```

cf. examples/example5.html

Note that the batch id – the second argument – has to be freshly created for your tests to actually work. make does that for you, including replacing the hex literal in the source code when telling it to run example 5:

[8] There are many ways to have an invalid IEEE 754 bit pattern, and different codes for "not a number" (NaN), depending on how it happened. POT JS simply takes what is in the float variable and uses the float format bit pattern. While they are represented the same ("NaN"), different NaN codes might not be the same bit pattern and thus not the same key.

[9] For an intro to Swarm, the concept of its network nodes, the Bees, and the postage batch IDs that power its incentive mechanisms, see https://ethswarm.org. You do not need to operate a Swarm Bee node to use POT JS: you can try it locally, with in-memory storage during development and testing. It can be operated with a local Swarm test network. But to write data to the decentralized Swarm mainnet, you will need a postage batch.

```
% make example5
```

Another example, for node.js (cf. **examples/example8.js**, pg. 65):

```
bee = process.argv[2]
batch = process.argv[3]
kvs = new pot.Kvs(bee, batch)
```

cf. examples/example8.js

An example invocation of this script snippet would be:

```
% node hello.js http://localhost:1633 \
 6792a183adafdac3a0a1a1e65b435406aff8f4343f980b16cabafd4a8525c0aa
```

Again, note that the batch id – the third argument – has to be freshly created for your tests, otherwise the **put()** or **save()** calls will fail, **currently with a misleading 404 error**. Use **make** to handle it.

TIMEOUT

All **put*()** and **get*()** methods, as well as **save()**, **load()**, and **delete()** that don't have **Sync** in their name accept an optional timeout argument, in milliseconds, that follows their main parameters:

```
try {
        await pot.ready()
        kvs = new pot.Kvs(beeUrl, batchId)
        await kvs.put("hello", "POT!", 1000) // 1 second timeout
        ref = await kvs.save(10000) // 10 seconds timeout
} catch(e) {
        ...
}

value = kvs.getSync("hello", 3000) // 3 seconds timeout
if(value instanceof Error) ...
```

The asynchronous functions' promise rejects when the timeout lapses and an error is thrown.

The synchronous functions (with **Sync** in their name) return an error, instead of any expected result, but don't throw an error.

When the timeout parameter is not given, or `null`, or `undefined`, the functions wait forever if they encounter a problem. This never happens in in-memory mode (when no URL and batch ID was given to the constructor or `load()`).

The timeout does not roll anything back but it will also not leave the KVS in an invalid state. It may race the completion of the function though. The server might just have sent the signal back that the function had been executed. There is no direct status report on that.

CANCEL

The methods `put*()`, `get*()`, `delete()`, `save()` or `load()` (that don't have a `Sync` in their name) can be canceled. The promise objects these functions return have `cancel()` methods.

The cancellation comes with no guarantees. The cancel might be late and not stop the complete execution of the original call. It merely means that the waiting should stop. It is possible that the awaited network response had just come in and the `put()`, `get()`, `delete()` etc. have just successfully been executed, racing the cancel. There is no status reported on that because the ok from the server might come in after the cancel has been executed. There is also no roll back aspect about cancel, it does not re-instate the prior state, it just gives up. However, it will not leave the KVS in an invalid state.

The way promises work, there are two caveats, as detailed below: **do not chain `.then()` and `.catch()` to the promise immediately**, that is, not before you assigned the promise to a variable. And avoid to inadvertently throw an uncaught rejection that stops your program.

METHOD CHAINING

Upon canceling, the promise rejects and if a `.catch()` method is chained, it is executed. But to chain `.catch()` and `.then()`, separate the chaining into its own statement like this:

```
promise = kvs.put("fox", "The quick brown fox jumps over the lazy dog!")

promise.catch(...)

...

promise.cancel()
```

Note the second line in the example. The `.catch()` cannot be chained immediately to the `put()` call, otherwise the `cancel()` call would not work.

Because chaining `.then()` or `.catch()` directly onto the `put()` call would return a different promise, namely the one which wraps the promise returned from `put()`, which is what happens in a `.catch()` or `.then()` but is usually syntactically transparent. The variable named '`promise`' would then contain the promise that `.catch()` wraped around the promise returned by `put()`. But this wrapping promise does not have the `cancel()` method. Therefore, the original promise that `put()` returns is the one to keep, to be able to cancel.

For clarity, this does **not** work, the `.catch()` must not be chained to the `put()` directly:

```
promise = kvs.put("fox", "The quick brown fox jumps!").catch(...)
...
promise.cancel() // TypeError: promise.cancel is not a function
```

TRY/CATCH

When `try/catch` blocks are used instead of `.catch()` then the `catch` block does **not** catch the rejection unless `await` is used on the promise in the `try` block.

It does not matter from where the `cancel()` is called, within or from out of the `try` block. It is not caught either way, if the thread has left the try block.

To illustrate, the rejection that this `cancel()` causes is **not** caught in the `catch` block.

```
try {
    promise = kvs.put("fox", "jumps!")
    promise.cancel()
} catch(e) {
    // the cancel() does not trigger this catch block
}
```

This causes an `uncaught rejection`. It can be caught by setting a global `unhandledRejection` handler. Which is better than having the program crash from the uncaught exception:

```
process.on("unhandledRejection", (error) => {  ...  }).
```

But it is not what you want to be in control of the flow.

The following `cancel()` **is** caught in the `catch` block, because `await` is used:

```
try {
    promise = kvs.put("fox", "jumps!")
    setTimeout(promise.cancel(), 1)
    await promise
} catch(e) {
    // the cancel() does trigger this catch block
}
```

For the example, to be able to use `await`, the try block has to be asynchronously decoupled from the `await`ed promise. A timeout that fires after one millisecond is used for that.

If a `put()` succeeds immediately, the `cancel()` might have no effect and not reject because the promise is already fulfilled when the cancel happens.

BATCH ID

All tests and most examples can be tried out **without** a batch ID: the **in-memory** storage mode works without it, and demonstrates its usefulness for learning, testing, and developing. But for production use you will likely want to store to Swarm mainnet. For that you will need a batch.

A batch ID stands for **postage stamps** that Swarm uses for incentivization, inter alia, to compensate its Bee nodes to keep copies of the stored data. Batch IDs are the prime payment mechanism that Swarm offers, pervading every aspect of decentralized data hosting on Swarm.

For trying out POT JS, it is **not** required to understand the concept of batch IDs, nor even for coding POT JS apps on a local Swarm testnet: there are `make` rules provided that set the local Swarm testnet up in a docker container and obtain a free batch id for your tests that works on the testnet.

You can easily glean from the `Makefile` how this test network is deployed and the free batch IDs acquired. This will help developing an app using Swarm without interacting with the Swarm mainnet.

The next page explains more about the local network.

But if you are planning to use the Swarm mainnet, it will be indispensable to understand the concept of batch IDs, how to buy them and for how much data and for how long they are good:

https://docs.ethswarm.org/docs/develop/tools-and-features/buy-a-stamp-batch

INSTALLING A LOCAL SWARM NETWORK

The installation and firing up of a local test network is automated, you only need docker installed and running to install and start:

```
% make locnet_install
% make locnet_start
```

As a batch ID is required when working with a network (instead of with the in-memory mode as most examples and tests do), this `locnet_start` rule creates a free postage batch ID and stores it in file `.batch_id`.[11]

All tests and most examples can be tried out **without** connection to any Swarm network because POT's in-memory mode allows for testing and developing without one. But to test an app with a Swarm network, the easiest way is to use the above `make` rules to install a local Swarm test network and run it.

The make rules use the docker-based installation by Fair Data Society:[10]

`https://www.npmjs.com/package/@fairdatasociety/fdp-play`

Alternatively, Swarm provides documentation on how to set up a local 2-node testnet:

`https://docs.ethswarm.org/docs/develop/tools-and-features/starting-a-test-network/`

As an example, the following script starts the Swarm test network, creates a new batch ID and stores it into the file `.batch_id`.[11] Generally, using `make` to initiate tests and examples does that for you. This script then stops the network in the last line. You may want to keep it running when developing.

```
# starting five local swarm nodes
fdp-play start --detach

# buy stamps (free in this test setup)
swarm-cli stamp buy --yes --verbose --depth 20 --amount 1b | tee .batch_creation
grep "Stamp ID:" .batch_creation | cut -c11-74 > .batch_id
echo "postage batch ID: $(cat .batch_id)"

# stopping nodes
fdp-play stop
```

cf. Makefile, rule node_locnet_test

The free batch IDs will only work on this local network and only for 2 days or until it is restarted.

---

[10] https://github.com/fairDataSociety/fdp-play – Discord https://discord.gg/KrVTmahcUA

[11] The file name starts with a dot and thus, by default, is not visible in Mac Finder, nor with `ls` in the terminal. Press `shift` + `command` + `.` or use `ls -a`.

EXPLICIT INITIALIZATION

The following might be helpful if a higher degree of control over the app startup sequence in the browser is needed.

For browser scripts (not for node.js), as an alternate way to initialize WASM, you may leave out **pot-web.js** and instead include only the generic Go WASM wrapper **wasm_exec.js**, from the **lib/** folder. Create an object called **Go** and instantiate the POT WASM executable **pot.wasm**, then run it once it is loaded. Cf. **examples/example3.html** (pg. 54).

```
const go = new Go()
WebAssembly.instantiateStreaming(fetch("lib/pot.wasm"), go.importObject)
   .then(async (r)=>{
          go.run(r.instance)
          kvs = new pot.Kvs()
   })
...
```

part of examples/example3.html

This way to initialize **pot.wasm** may have benefits in some scenarios. For a more detailed discussion of it, see Initialization, pg. 98. Most of the time, using **pot-web.js** will be the right choice.

SYNCHRONOUS INITIALIZATION

Especially for synchronous scenarios, you can write a function called `onPotInitialized()` to kick of your app, instead of **awaiting** **pot.ready()**. This function is called by the WASM executable when initialization is complete, i.e. when **pot.ready()** would resolve. Before that function is called, the **pot** object might not be complete and be missing methods. Cf. **examples/example4.html** (pg. 58).

```
function onPotInitialized() {
    kvs = new pot.Kvs()
    kvs.putSync("hello", "POT!")
    value = kvs.getSync("hello")
 }
```

part of examples/example4.html

VERBOSITY

The verbosity log level controls how much output to the console or terminal is generated by POT JS. The log prints to screen when running POT JS with node.js. In the browser, it logs into the browser console. The log level can be changed using `setVerbosity()`.

The default log level is `DEBUG`, printing a fair amount of details of processing steps and state change to the browser console or screen.

0	`pot.NONE`	no logging
1	`pot.CRITICAL`	logs only errors that appear to arise from a malfunction of POT JS.
2	`pot.ERROR`	programming errors stemming from wrong use are also logged.
3	`pot.INFO`	general runtime information is logged.
4	`pot.DEBUG`	state changes, like binary level put and get keys and values are logged.
5	`pot.TRACE`	Some code is wired to report on every step of the way.

Log Levels

Normal operation will benefit from the `INFO` log level. `DEBUG` can help to find confirmation about suspicions while hunting programming errors. `TRACE` can make sense when extending and optimizing POT JS itself. `NONE`, `CRITICAL` or `ERROR` silence all logging from POT JS so that the focus can be put elsewhere.

The default log level that a call to `pot.log()` uses is `INFO`. This means that a call to `pot.log()` will show up in the logs unless verbosity of the system is set to `ERROR` or lower. An arbitrary log level can be passed to `pot.log()` as optional second parameter.

Regarding verbosity of tests, see pg. 80 and 85.

The log level affects output to both standard output and standard error. At `ERROR` and `CRITICAL`, no logging to standard output takes place, but only to standard error.

Note that suppressing the logging of errors does not mean that the errors are not being thrown (or in the case of `*Sync()` functions, returned instead of the expected return value). The logging to standard error happens in parallel to the error handling expected of your app.

Log levels can be combined with detail flags. The flags are ORed to the level. See below.

2048	`pot.MEMORY`	log heap size information (gold on screen)
4096	`pot.NOCUT`	log hex digits in full length instead of abbreviated

Log Level Flags

SETTING THE LOG LEVEL

The verbosity can be adjusted with `pot.setVerbosity()`. The call returns the previous setting. The current setting is returned by `pot.getVerbosity()`.

```
(async () => {
    await pot.ready()
    pot.setVerbosity(pot.INFO)
    ...
```

part of examples/example8.js

To add a log level flag, it is ORed with the log level. E.g., to add heap size information output:

```
    pot.setVerbosity(pot.INFO | pot.MEMORY)
```

ALTERNATE, EARLIER SETTING

If it is important to suppress even the first log line coming from POT JS, or to set a log level from the very first moment, the following are the ways to set verbosity before the WASM executable is even initialized. These ways to set verbosity need a literal **integer** supplied, because the `pot.*` constants listed above do not exist yet at that point. So, to set `pot.ERROR` log level, use 2 etc.

- Setting `globalThis.potVerbosity`. This can be the most robust way.

```
var go = new Go()
global.potVerbosity = 3
WebAssembly.instantiate(fs.readFileSync("lib/pot.wasm"), go.importObject)
    .then((r) => { go.run(r.instance) })
```

part of examples/example8.js

- Adding the attribute `verbosity` to the script tag of `pot-web.js` in the browser:

```
<script src="potjs/lib/pot-web.js" verbosity=2></script>
```

part of tutorial/t4.html

- Adding it to the require statement as second or third parameter, **in separate brackets**:

```
require("./potjs/lib/pot-node")(0)
```

part of tutorial/t9.js

```
require("./potjs/lib/pot-node")("lib2/pot.wasm", 5 | 4096)
```

part of tutorial/t9.1.js

ERROR PROPAGATION

The error strategy of POT JS is Javascript-style to the degree possible, generally using JS exceptions rather than returning error codes as is the Go style. But for the `*Sync()` functions, an error is returned instead of thrown due to a technical limitation of the Go/Javascript interface.[12] That means that for synchronous functions, return values have to be tested for an error where normally an exception would be expected.

CRASHES

There is a potential for deadlocks and pseudo deadlocks emanating from POT JS's hybrid nature that the advanced user should be aware of. If a deadlock happens, the WASM executable crashes and all state that it caches is lost. If a reference of a KVS has not been retrieved with `save()`, it cannot be obtained after the crash. The easy way to minimize such problems is to **avoid synchronous functions when working in the browser**.

If the Go WASM executable, `pot.wasm`, deadlocks, or 'concludes' that it might be deadlocked, it crashes and cannot be recovered. The transient state of all KVS objects is lost, all POT JS functions stop working and an error is thrown instead when calling them that reports that the Go executable is down, and/or that a method of a KVS (that previously existed), does not exist now. In this case, `pot.wasm` must be re-initialized before the POT JS functions can be used again. This affects all `pot` and `kvs` functions. It means that while everything else might still work, the *methods* of a `kvs` that on the face looked like native Javascript functions, are just 'gone' when the WASM executable that provided them has crashed.

The reasons for such crashes are real deadlocks or suspected deadlocks flagged by the Go runtime, as well as uncaught runtime errors – 'panics' – on the side of the Go executable. The capacity of deadlocks is unescapable for a WASM architecture because of the fact that Javascript and Go WASM share a single thread. Since both runtimes do not have full visibility, the Go runtime deadlock detection can throw false positives.[13] Multiple measures are in place for POT JS to keep the WASM

---

[12] The implementation of POT JS is mainly in Go (`potjs.go`), not in Javascript. JS code is created in Go, for full data/situational access on the Go side (cf. pg. 85). A major limitation of the Go/Javascript interface `syscall/js` is that Javascript exceptions cannot be thrown from Go. What works is to call the `reject` parameter of in the executor of Javascript promises that are constructed in Go. Because the more relevant functions are promise-based at any rate, this covers the common use cases.

[13] Javascript does not have a deadlock detection. It does not need one because it is not made for multi-threading. Its asynchronicity is a programming paradigm that feels parallel but is implemented on one thread, shielding the programmer from the more involved challenges of true parallel execution. Go is built to operate on multiple threads in parallel and one can therefore, relatively easily, create real deadlocks in Go. As Go, too, is protective in nature, it comes with an automatic deadlock detection. But this protection cannot operate well when it cannot 'see' where the main thread is going, when control is really with the main Javascript thread, in Javascript land, with nothing going on in Go. To the deadlock detection this might look as if things had stalled. This friction by necessity leaks into POT JS.

executable alive in all scenarios, to handle panics gracefully and to not allow a deadlock to happen. To maximize robustness, avoid synchronous functions in the browser for production. This is best practice at any rate, so as to not freeze the browser while waiting for a server response.

FURTHER READING & SUPPORT

For implementation details, check out the developer notes from page 92.

The **tutorial** follows, presenting a number of relevant angles to get a fuller picture. Find tutorial code snippets in the `tutorial/` folder.

Refer to the examples in the `examples/` folder and the **examples** chapter (pg. 48) to understand how POT JS might be used in the specific use case you envision.

There is a function overview, pp. 47. The separate **function reference** document in the `doc/` folder explains all available functions with example snippets.

The source of POT JS is hosted at `https://github.com/brainiac-five/potjs`.

The Go POT implementation is hosted at `github.com/ethersphere/proximity-order-trie`.

For more information on Swarm, visit `https://ethswarm.org`.

How to obtain postage batches is described at `https://www.ethswarm.org/get-bzz`.

Find support at `https://discord.ethswarm.org`.

The capability to deadlock is not a fault of the Go POT (proximity-order-trie) implementation but lies in the single-thread nature of Javascript that can be programmed highly asynchronously but is not truly multi-threaded, except when using the duly limited Web Workers. That Javascript is designed this way provides strong safeguards for the programmer against a class of difficult to track errors. But the guardrails are coming off when crossing from Javascript into Go-land *and back*: a blocking wait in Go that hands control back to the browser or node.js deadlocks its main Javascript execution thread. Javascript waits for Go to continue and Go for Javascript. This is reliably the case when Go executes a network fetch that is internally handed back from WASM to Javascript to execute. When using synchronous POT JS methods to write to and read from Swarm, almost every function may use a fetch. When using one of the in-memory persistence models, none does.

TUTORIAL

This is a step-by-step guide to learn how to use POT JS.

This tutorial discusses the core building blocks, alternate options, and some special aspects that are good to know. The code examples used and referred to in the captions are in the `tutorial/` folder. If you do as instructed below, this folder will be at . /`potjs/tutorial` from your working directory.

We will start out with a minimal node.js example, then try the same code in the browser, develop it into a primitive web app, then shift the resulting functionality back from the browser to the server-side, to work with node.js again. Finally, we'll use Docker to run a local Swarm test network and check out how connecting to a real storage network works.

Overall, the matter is simple. But it will help to get a bit of background, too, as provided below. If you like it faster, check out the `examples/` folder. It may be all you need to get going. The examples are explained from pg. 48; then use the `function reference` document in the `doc/` folder.

INSTALLATION

All of the following works with node.js 22.17.1 on a Mac.

As a workspace for this tutorial, create a new directory, and clone POT JS into it from github:

```
mkdir tut
cd tut
git clone https://github.com/brainiac-five/potjs
cp potjs/favicon.ico .
```

There is no building or installation required beyond this.

The relevant files we'll be using are:

- `potjs/lib/pot.wasm`
- `potjs/lib/pot-web.js`
- `potjs/lib/pot-node.js`

These contain the Go implementation of POT with a Javascript interface compiled to WASM – `pot.wasm` – plus the Javascript initialization code for it, for in-browser and node.js respectively. The files are portable across operating systems and browsers and run on Mac and Ubuntu terminals.

TUTORIAL #1 – HELLO, P.O.T.!

The main functions of POT JS are **put()** and **get()**. Key-values-stores (KVS) are created with **new**. They are saved to the network or to an in-memory storage by **save()** and retrieved with **load()**.

Starting out, we will show how **new**, **put()** and **get()** work.

THE CODE

For our first file, in **tut/** – the folder you just cloned potjs into –, create a file **t1.js** and paste this:

```
require("./potjs/lib/pot-node")

; (async () => {
      await pot.ready()
      kvs = new pot.Kvs()
      await kvs.put("hello", "P.O.T.")
      value = await kvs.get("hello")
      console.log("hello:", value)
})()
```

tutorial/t1.js

You also find this script in **potjs/tutorial/t1.js**.

Let's talk through it. The first line **requires** the Javascript wrapper **lib/pot-node.js** around the WASM executable **lib/pot.wasm**, which also initializes it. **pot.wasm** is a Go program compiled to WASM. The main source file is **./potjs.go**. After **pot.wasm** is loaded, it is started and kept running. It creates Javascript objects and fleshes out the main global object **pot**.

```
require("./potjs/lib/pot-node")
```

In the odd case that **pot.wasm** was not in the same folder as **pot-node.js**, an additional argument to require can be used. This extra parameter is not placed within the brackets but gets an extra set of brackets (see pg. 7).

The fourth line blocks until the initialization of **pot.wasm** is complete. That is, **pot.wasm** is loaded and its main function is done with creating the **pot** object. That's when POT JS is ready to go.

```
await pot.ready()
```

The loading and starting of **pot.wasm** happens asynchronously. The Javascript object **pot** is created early on in pot-node.js. But **pot.wasm** assigns the methods to **pot**. This is how the interfacing between Go and Javascript works: functions that are written in Go transparently become methods of Javascript objects. Therefore, before **pot** functions like **pot.Kvs()** can be used, the initialization of **pot.wasm** needs to be complete. That's what **pot.ready()** waits for.

The next line creates a key-value store (KVS).

```
kvs = new pot.Kvs()
```

Because we call **pot.Kvs()** without any parameters, the KVS will be stored in memory, not to a network. We will stick with this easier mode for a moment, while we are exploring the basics.

Next, we store a value to the KVS – "**P.O.T.**" – under the key "**hello**":

```
await kvs.put("hello", "P.O.T.")
```

We'll read it back out from the KVS:

```
value = await kvs.get("hello")
```

And after that the value, **P.O.T.**, is logged to the console.

RUNNING IT

To run this script, do:

```
node t1.js
```

WHAT YOU SHOULD SEE

```
pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  › created new in-memory persister
pot:  › new slot 1
pot:  › slot ref: 1 6488f1a48dad51f4b496714c7df1a17f…
pot:  » put  hello: P.O.T.
pot:  › →  68656c6c6f000000000000000000000000…: 03502e4f2e542e
pot:  » get  hello: P.O.T.
pot:  › ←  68656c6c6f000000000000000000000000…: 03502e4f2e542e
hello: P.O.T.
```

IN-MEMORY STORAGE

The default log level setting of POT JS is **DEBUG** level, so POT JS shares some internals. See pg. 18.

In line 5 of the log, POT JS tells us that it is 'persisting' the key-value store in memory.

```
pot:  › created new in-memory persister
```

POT JS behaves and is programmed the same whether it persists to memory or a network. The latency of method calls will be higher and there are devices to fine-tune. We will come back to that.

When creating a system using POT JS, at some point on will be more convenient to develop with a network storage attached because it will keep everything stored across program restarts. The in-memory storage on the other hand starts with a clean slate every time a script is restarted. For an earlier phase this will be more convenient.

BINARY REALITY

Line ten of the log shows the binary form of what we are storing. The code

```
await kvs.put("hello", "P.O.T.")
```

produces as log output

```
pot:  › →  68656c6c6f00000000000000000000000…: 03502e4f2e542e
```

In some cases, checking the binaries of what is really getting persisted (like **68656c6c6f0000…**) can be helpful.

Keys can be **strings**, **numbers** or **Uint8Arrays**. What is going on underneath is: the key, **hello**, is stored as **68656c6c6f0000…** (with a couple more zeros) because Javascript uses UTF-16 and **68 65 6c 6c 6f** is the hex (ASCII and) UTF-16 code for **hello**. A key is then zero-padded to 32 bytes on the binary level (for more details on keys see pg. 9). 32 bytes is also the maximal length of a key. Trying to use a key that is longer, causes an error to be thrown. **s.length** returns the Unicode units of a string – which is not necessarily its lengths in bytes. But you can use **pot.byteSize(s)** to find out the byte-size of a Javascript string. And **pot.truncString(s)** can be used to cut a string down to a required byte-size. E.g., 32.

P.O.T. becomes **03502e4f2e542e**. The byte representation of **P.O.T** is prepended with a type byte: **03** stands for **string**. **50 2e 4f 2e 54 2e** is UTF-16 (and ASCII or ANSI) for **P.O.T.** A value is not padded and can be 100,000 bytes – or much longer (see pg. 9).

Values can be **strings**, **numbers**, **boolean**, **Uint8Arrays** or **null**. **Undefined** is returned for keys that have no value stored in the KVS. **BigInt**, **symbols**, or **objects** are not supported (pg. 9).

TUTORIAL #2 – MOVING TO THE BROWSER

The same code, pretty much, runs in the browser. Copy **t1.js** to **t2.html**.

```
cp t1.js t2.html
```

Replace the first line with the lines

```
<script src="lib/pot-web.js"></script>

<script>
```

If **pot.wasm** were not in the same folder as **pot-web.js**, a path tag can be used (see pg. 7).

To the end of the file add:

```
</script>
```

The file should now look like **potjs/tutorial/t2.html**:

```
<script src="potjs/lib/pot-web.js"></script>

<script>
; (async () => {
        await pot.ready()
        kvs = await pot.new()
        await kvs.put("hello", "P.O.T.")
        value = await kvs.get("hello")
        console.log("hello:", value)
})()
</script>
```

tutorial/t2.html

RUNNING IT

Run the file by using **http-server** or any other tool to serve the current directory. Note that the browser will, intentionally, show a blank page.

```
npx http-server -c1 -o t2.html .
```

The -c1 parameter sets the cache to 1 second, the minimum. -o opens the file in a browser.

Don't trip yourself up with the browser's page cache when experimenting.

WHAT YOU SHOULD SEE

In the browser, nothing obviously, an empty page. But if you open the browser console:

BROWSER CONSOLE

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  › created new in-memory persister
pot:  › new slot 1
pot:  › slot ref: 1 9b4b8a19042abb3b1d023eab79f47b57…
pot:  » put  hello: P.O.T.
pot:  › →  68656c6c6f000000000000000000000000…: 03502e4f2e542e
pot:  » get  hello: P.O.T.
pot:  › ←  68656c6c6f000000000000000000000000…: 03502e4f2e542e
hello: P.O.T.
```

This is the same log as above, except for the second line, **`browser detected`**.

BROWSER VS NODE.JS

It is relevant for POT JS whether node.js or a browser is its sandbox, because the way that network access works. Counter-intuitively, a WASM executable running in the browser has better access. Generally speaking, WASM was developed for the browser, and a browser script has a sandbox around itself that protects the machine. A WASM executable executed in a node.js environment does not have that sandbox around it and therefore its network access is curtailed directly.

TUTORIAL #3 – SUB-COMPONENT INTEGRITY

To safeguard the integrity of the included script, fingerprints can be hardcoded into HTML.

The required SHA-384 hash can be found in **`potjs/lib/pot-web.js.sha384`**. It looks similar to this:

```
% cat potjs/lib/pot-web.js.sha384

Yb+faMKZRYWBwk2CrH9NeKRzg14w79Hyn7WCLdQbreD7coOaE+17RHMaPJv4zviS%
```

The hash you see will differ, depending on the version of POT JS that you are using. The **`%`** at the end is not part of the hash.

There are pages online where you can verify the SHA-384.[14]

[14] E.g., https://emn178.github.io/online-tools/sha384.html

Copy `t2.html` to `t3.html` and add the `integrity` attribute. `"sha384-"` must be prepended to the hash.

The first line becomes (with or without line break, and with the right hex digits for your edition):

```
<script src="potjs/lib/pot-web.js"
integrity="sha384-Yb+faMKZRYWBwk2CrH9NeKRzg14w79Hyn7WCLdQbreD7coOaE+17RHMaPJv4zviS">
```

head of tutorial/t3.html

The file should now look like `potjs/tutorial/t3.html`. The hash in file `tutorial/t3.html` will be up-to-date. That is, matching the file `lib/pot-web.js.sha384`.

The code is run and works like the example 2 before, showing the same results.

```
npx http-server -c1 -o t3.html .
```

TUTORIAL #4 – SAVE, LOAD, AND DELETE

We will build on this example and discuss `save()`, `load()`, and `delete()`.

```
cp t3.html t4.html
```

We'll swap the first line out for a script tag that this time includes a verbosity attribute. We choose verbosity 2, which is normal `ERROR` level of log details. Unless there is an error, POT JS will produce no log lines.

```
<script src="potjs/lib/pot-web.js" verbosity=2></script>
```

part of tutorial/t4.html

We will add a number of lines after `console.log("hello:", value)` that save, load, and delete values. Here are the first two:

```
ref = await kvs.save()
console.log("save reference:", ref)
```

part of tutorial/t4.html

These lines `save` the key-value cache to storage. In the instance this does not permanently persist the `put()` that came before, because the storage of this example is in-memory, and not a network. It will not persist when the program is stopped. But these lines will return the **save reference**, which is the root of the proximity order trie. The save reference is how a complete KVS can be re-accessed with `load()` at a later point. For in-depth info on the save reference, see pg. 9.

The save reference is the root of the state trie of the KVS. it changes with every change to the KVS.

That is, if we update the KVS, the save reference will have changed, too. We will check that out:

```
await kvs.put("Hogy vagy", "Viktor")
value = await kvs.get("Hogy vagy")
console.log("Hogy vagy:", value)

ref = await kvs.save()
console.log("save reference:", ref)
```

part of tutorial/t4.html

The above lines `put` an additional key-value pair to the KVS – `Hogy Vagy`, `Viktor` – and save the KVS again. This will display a different save reference, because the root hash of the POT has changed due to the new key-value pair having become part of it.

We'll now check out how deletion of a key-value pair works.

```
await kvs.delete("Hogy vagy")
value = await kvs.get("Hogy vagy")
console.log("Hogy vagy:", value)
```

part of tutorial/t4.html

Deleting this value leads to an `undefined` value being returned as response to an attempt to `get` it. The same as for keys that never had a value assigned. Such a deleted value is unlinked from the trie.

But being stored on an immutable network – that's in the name – it is not deleted from the storage network itself. However, Swarm does discard values at some point: when the postage expires, all linked resource can be garbage collected by the Bee nodes that store them.

```
ref = await kvs.save()
console.log("save reference:", ref)
```

part of tutorial/t4.html

Saving and inspecting the save reference again, you will, when we run this example, see that the reference is the same again now like the first time around. Because we are back to the exact same state as it was after the first put.

RUNNING IT

Run this example like the examples before:

```
npx http-server -c1 -o t4.html .
```

WHAT YOU SHOULD SEE

BROWSER

An empty page.

BROWSER CONSOLE

We reduced the POT JS output to `ERROR` level, so we will only see the `console.log()` calls.

Observe how the save reference is the same when the KVS contains the same key-values.

```
hello: P.O.T.
save reference: b77d8826af7c1684ae88bb63afdee7e3b81bcb6d0ee15a4d1aa096452dcb7624
Hogy vagy: Viktor
save reference: 830c497a541c8d0993718435bf20746f8c22e89fc5c9ba61de5f0774dd7c1838
Hogy vagy: undefined
save reference: b77d8826af7c1684ae88bb63afdee7e3b81bcb6d0ee15a4d1aa096452dcb7624
```

TUTORIAL #5 & 6 – INTERACTION

We will stay with the browser and look at an interactive example. Create file `t5.html`.

```
<pre>

    key     <input id=key>
    value   <input id=value>
            <button onclick="put()"> put </button>

</pre>

<script src="potjs/lib/pot-web.js"></script>
<script>

    var kvs

    (async () => {
            await pot.ready()
            kvs = new pot.Kvs()
    })()
```

```
    async function put() {
            key = document.getElementById('key').value
            value = document.getElementById('value').value
            await kvs.put(key, value)
    }
</script>
```

tutorial/t5.html

This source is also filed under **potjs/tutorial/t5.html**.

The first lines are a web form that allows to input a key and a value. When the put button is clicked, the **put()** function further down is called.

```
<pre>

 key    <input id=key>
 value  <input id=value>
        <button onclick="put()"> put </button>

</pre>
```

part of tutorial/t5.html

Then comes the POT initialization, where a KVS is created.

```
<script src="potjs/lib/pot-web.js"></script>
<script>

 var kvs

 (async () => {
    await pot.ready()
    kvs = new pot.Kvs()
})()
```

part of tutorial/t5.html

Note that we are still operating in-memory: the pot.Kvs() constructor does not get any network parameters.

Then a **put()** function that reads from the form and stores the key-value pair to the KVS we created.

```
async function put() {
   key = document.getElementById('key').value
   value = document.getElementById('value').value
   await kvs.put(key, value)
}
```

part of tutorial/t5.html

RUNNING IT

Run the page with

```
npx http-server -c1 -o t5.html .
```

WHAT YOU SHOULD SEE

BROWSER

key

value

put

BROWSER CONSOLE

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  › created new in-memory persister
pot:  » new slot 1
pot:  › slot ref: 1 a852dc20656ddf85ede5bece64aafbf4…
```

▶ Entering input for key and value, and hitting put:

BROWSER

key foo

value bar

put

BROWSER CONSOLE

```
...
pot:  » put  foo: bar
pot:  › ⟶  666f6f000000000000000000000000…: 03626172
```

▶ We will now add the **get**ting, analogous to the **put**.

To the form, add (before the **</pre>** tag):

```
<hr />

 key    <input id=search>
        <button onclick="get()"> get </button>
 value  <input id=result>
```

part of tutorial/t6.html

Add this **get()** function:

```
 async function get() {
    key = document.getElementById('search').value
    value = await kvs.get(key)
    document.getElementById('result').value = value
 }
```

part of tutorial/t6.html

The file should now look like **potjs/tutorial/t6.html**:

```
<pre>

 key    <input id=key>
```

```
  value  <input id=value>
         <button onclick="put()"> put </button>

<hr />

  key    <input id=search>
         <button onclick="get()"> get </button>
  value  <input id=result>

</pre>

<script src="potjs/lib/pot-web.js"></script>
<script>

  var kvs

  (async () => {
     await pot.ready()
     kvs = new pot.Kvs()
  })()

  async function put() {
     key = document.getElementById('key').value
     value = document.getElementById('value').value
     await kvs.put(key, value)
  }

  async function get() {
     key = document.getElementById('search').value
     value = await kvs.get(key)
     document.getElementById('result').value = value
  }

</script>
```

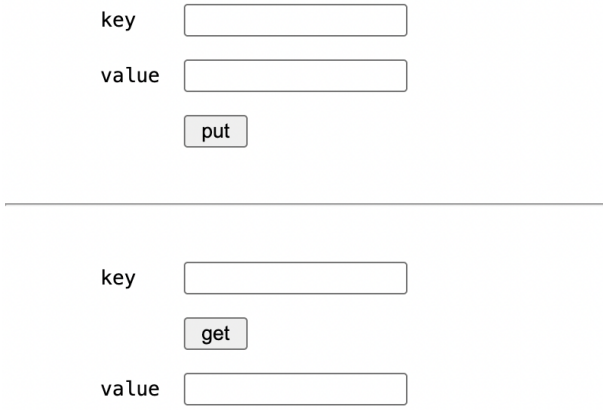
tutorial/t6.html

RUNNING IT

```
npx http-server -c1 -o t6.html .
```

WHAT YOU SHOULD SEE

BROWSER



BROWSER CONSOLE

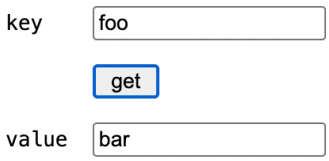
As before.

▶ Entering **foo**, **bar** and putting it to the KVS will also show the same browser console log as before:

```
...
pot:  » put  foo: bar
pot:  › ⟶  666f6f000000000000000000000000000…: 03626172
```

▶ Entering **foo** in the 3rd input field labeled **key** and hit **get**, will cause the **value** field to display **bar**:

BROWSER



BROWSER CONSOLE

```
...
pot:  » get  foo: bar
pot:  › ⟵  666f6f000000000000000000000000000…: 03626172
```

AVOIDING DEADLOCKS

A WASM executable is technically not a library but a Go program that runs continually, waiting for the next request. Unfortunately, if it crashes, the functionality it extends and state it holds goes away. The global **pot.*** functions and every **kvs.*** method that POT JS defines become unavailable. From the Javascript perspective, they just don't exist anymore.

It is relatively easy to deadlock the WASM executable by using the synchronous functions in the wrong context. Those are the POT JS functions ending on **Sync**. Specifically, **putSync()**, **getSync()**, **deleteSync()**, and **saveSync()** crash the WASM executable by deadlock when using a Swarm network as storage backend in the browser. A deadlock error looks like this, followed by a long stack trace:

```
fatal error: all goroutines are asleep - deadlock!
```

a deadlock error

This is why these synchronous functions are returning an error when it is attempted to use them with a network in the browser. This error protects the WASM from crashing.

```
### error in put*(): no sync calls to networks in-browser
```

a 'preventative' error

The synchronous functions work well for any in-memory experimenting, and in any case with node.js.

In case the Go WASM executable goes down, it has to be re-initialized. **pot-node.js** and **pot-web.js** do not currently offer functions to help with the re-initialization. But their main functions can serve as blueprints for how it works. The clearest path can be the alternate way of initializing explained for example 3 (pg. 54):

```
<script src="lib/wasm_exec.js"></script>
<script>

 var kvs
 const go = new Go()
 window.potVerbosity = 3
 WebAssembly.instantiateStreaming(fetch("lib/pot.wasm"), go.importObject)
     .then(async (r)=>{
            go.run(r.instance)
            kvs = new pot.Kvs()
     })
```

part of examples/example3.html

TUTORIAL #8 – VERBOSITY

The default logging to screen or console is relatively detailed. If you want to see less or nothing at all, the level of logging can be changed with a value from 0 to 5; zero meaning no logging at all, 4 what you saw in the terminal up to now. The levels have names:

#	name	description
0	pot.NONE	no logging, not even errors.
1	pot.CRITICAL	logs only errors that appear to arise from a POT JS malfunction.
2	pot.ERROR	logs all programming and runtime errors.
3	pot.INFO	general runtime information is logged, close to what you saw so far.
4	pot.DEBUG	certain data, like binary keys and values are logged. The default.
5	pot.TRACE	some very specific program paths and minor settings are logged.

The verbosity level is set using **pot.setVerbosity()**.

To try it, copy **t6.html** to **t8.html** and add the **setVerbosity()** call to **INFO** after **pot.ready()**:

```
(async () => {
    await pot.ready()
    pot.setVerbosity(pot.INFO)
    kvs = await pot.new()
 })()
```

part of tutorial/t8.html

RUNNING IT

```
npx http-server -c1 -o t8.html .
```

▶ Trying the same **puts** and **gets** as before (**foo-bar** put, **foo** get), the low-level log lines are gone.

WHAT YOU SHOULD SEE

BROWSER CONSOLE

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  » created in-memory persister
pot:  » new slot 1
pot:  » put  foo: bar
pot:  » get  foo: bar
```

log of tutorial/t8.html – with verbosity setting pot.INFO

If you look closely, the lines that had a single angle › are now not logged any more. They were DEBUG level. The ones with a double angle » remain. They are INFO level.

On the other hand, to get *more* information you can add **log level flags**, see pg. 18.

The TRACE level is useful only for developing and contributing to POT JS.

▶ Setting verbosity to pot.NONE results in this output.

Four lines remain, even with pot.NONE:

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
```

log of tutorial/t8.1.html – verbosity setting pot.NONE

To eliminate those remaining four lines, a different method has the be used: a verbosity attribute can be added to the script tag that designates pot-web.js:

```
<script src="potjs/lib/pot-web.js" verbosity=2></script>
```

part of tutorial/t8.2.html

This verbosity setting suppresses even the first four lines coming from POT JS.

The value assigned to the verbosity attribute must be a number (or a variable you defined) because the constants pot.NONE etc. do not exist yet at this point. The initialization of WASM creates them.

If you try it out, there will be no logging now in the browser console while you put and get values. Setting "2," meaning ERROR, is enough to silence all logging, except for errors. A lower level than that would suppress even error logging. However, errors would still be thrown in your program (or returned, for *Sync() functions), just not logged automatically.

For a node.js script to likewise set the verbosity at the earliest an additional parameter can be added to the require call. Copy t1.js to t9.js and add (0) to the require statement.

```
require("pot-node")(0)
```

part of tutorial/t9.html

To run the script:

```
node t9.js
```

What you should see on the console should again be only what the **t9.js** script prints to it.

```
hello: P.O.T.
```

In the rare case that there is also a separate path for **pot.wasm** in the mix, the verbosity value moves into the second place of the second bracket. An example looks like this:

```
require("./potjs/lib/pot-node")("wasm/pot.wasm", 5 | 4096)
```

part of tutorial/t9.1.html

To try it, create **potjs/lib/wasm**, move (not copy) **pot.wasm** there and run **node t9.js** with the changed require line as above. (Move **pot.wasm** back after.)

You'd see a lot of output, because 5 is for **TRACE** and **| 4096**, for **NOCUT**, spells hex digits out in full.

```
jsi:  • path argument :  ./lib2/pot.wasm
jsi:  • resulting path:  /Users/b5/tutxx/potjs/lib/lib2/pot.wasm
jsi:  • pot wasm initialization
jsi:  • pot.ready initialized
pot:  800  verbosity set 4101
pot:  803  » POTWASM
pot:  803  » node.js detected
pot:  816  » init done
pot:  816  » ready
pot:  817  • create basic context
pot:  817  » created new in-memory persister
pot:  817  » new slot 1
pot:  817  • created 32 byte reference
 ‹6268555cb71ce569927dedaae94ef9f3471d2c220b85c9fec4769782ce6f5c7f›
pot:  818  › slot ref: 1
 6268555cb71ce569927dedaae94ef9f3471d2c220b85c9fec4769782ce6f5c7f
pot:  819  • create cancellable context
pot:  821  » put  hello: P.O.T.
pot:  821  › →
 68656c6c6f000000000000000000000000000000000000000000000000000000: 03502e4f2e542e
pot:  821  • create cancellable context
pot:  822  » get  hello: P.O.T.
pot:  822  › ←
 68656c6c6f000000000000000000000000000000000000000000000000000000: 03502e4f2e542e
hello: P.O.T.
```

Regarding verbosity of test suites, see pg. 80 and 85.

Note that in the terminal, when using POT JS with node.js, errors are logged to standard error.

TUTORIAL #11 – NODE APP

We will now briefly look at a primitive, single-script node.js web app, which works like the form we created in chapter #5 & 6, with the difference that the POT JS logic now runs on the server instead of in the browser. That means, as similar as it looks to #5 & 6, this code now is executed by node.js instead of in the browser.

Specifically, the POT JS code is as small as before. There are no surprises. Except – that POT JS logs to the terminal, not the browser console.

The script looks like this. We will talk through it immediately below.

```
const http = require("http")
const qs = require("querystring")
require("./potjs/lib/pot-node")

const form = `<!DOCTYPE html><head><meta charset="UTF-8"></head>
            <pre>

            <form method=post action="http://localhost:3000">

            key    <input name=key>
            value  <input name=value>
                   <input type=submit name=button value=put>

      <hr />

            key    <input name=search>
                   <input type=submit name=button value=get>
            value  <input name=result>

            </form>

      </pre>`

var kvs

; (async () => {
      await pot.ready()
      kvs = new pot.Kvs()
})()

async function put(key, value) {
      await kvs.put(key, value)
      return `put ${key}: ${value}`
}

async function get(search) {
      value = await kvs.get(search)
```

```
        return `get ${search}: ${value}
                <script>
                document.getElementsByName('result')[0].value = '${value}'
                </script>`
}

const server = http.createServer(function(request, response) {

        if (request.method == 'POST') {

                var body = ''
                request.on('data', function(data) {
                        body += data
                })

                request.on('end', async function() {

                        var log
                        const post = qs.parse(body)

                        if(post.button == 'put')
                                log = await put(post.key, post.value)
                        else
                                log = await get(post.search)

                        console.log("srv: " + log.match(/.*/)[0])

                        response.writeHead(200, {'Content-Type': 'text/html'})
                        response.end(form + '<hr><br><pre>\t\t' + log)
                })
        }
        else {
                response.writeHead(200, {'Content-Type': 'text/html'})
                response.end(form)
        }
})
.listen(3000, '127.0.0.1')

console.log(`srv: listening at http://127.0.0.1:3000`)
```

tutorial/t11.js

RUNNING IT

```
node t11.js
```

WHAT YOU SHOULD SEE

At startup, this is logged to the **terminal**.

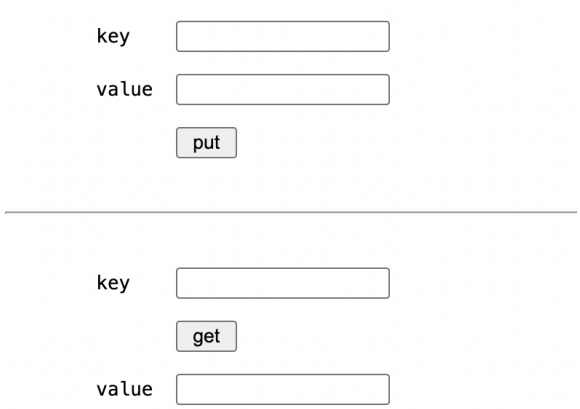
TERMINAL

```
srv: listening at http://127.0.0.1:3000
pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  › created new in-memory persister
pot:  » new slot 1
pot:  › slot ref: 1 84b7797ef4e0f9d928a943d6e7279146…
```

BROWSER

Now we'll let this serve up a page to the browser.

▶ Open http://127.0.0.1:3000 (you have to open the URL yourself this time) you should see the same picture as in the previous tutorial:



screen of tutorial/t11.js – http://localhost:3000

▶ Entering the same input – **foo**, **bar**, put; **foo** get – should result in these familiar logs:

```
pot:  » put  foo: bar
pot:  › →  666f6f000000000000000000000000000…: 03626172
srv: put foo: bar
pot:  » get  foo: bar
pot:  › ←  666f6f000000000000000000000000000…: 03626172
srv: get foo: bar
```

POT JS behaves the same, whether it is running in the browser or on the server.

TUTORIAL #12 – PERSISTENCE

To connect with a network, so the KVSs are stored to it, the Kvs constructor has to be supplied with a URL to a Swarm bee node, and a batch ID:

```
kvs = await pot.new(swarm_url, batch_id)
```

To be able to continue with a persisted KVS, at some point, there should be a **save()** to get the save reference for it. We put it immediately after the **put()**:

```
await kvs.put("hello", "P.O.T.")
await kvs.save()
```

We'll read the Swarm URL and batch ID from the command line (lines 5 and 6).

The resulting script is in tutorial/t12.js:

```
require("./potjs/lib/pot-node")

; (async () => {
        await pot.ready()
        swarmUrl = process.argv[2]
        batchId = process.argv[3]
        kvs = new pot.Kvs(swarmUrl, batchId)
        await kvs.put("hello", "P.O.T.")
        await kvs.save()
        value = await kvs.get("hello")
        console.log("hello:", value)
})()
```

tutorial/t12.js

LOCAL SWARM NETWORK

For our tests, we are setting up a local Swarm test network on Docker and connect to that. Not least because we don't need real BZZ tokens for that.

▶ Install the Swarm CLI, and the test network provided by Fair Data Society.

```
% npm install -g @ethersphere/swarm-cli
$ swarm-cli --version
$ npm install -g @fairdatasociety/fdp-play
$ fdp-play --version
```

▶ Start Docker and then spin up the network:

```
% fdp-play start --detach
```

You should see:

```
✓ Network is up
✓ Blockchain node is up and listening
✓ Queen node is up and listening
✓ Worker nodes are up and listening
```

▶ Next, we 'buy' some (test) Swarm postage stamps – the **batch ID** – using the prepopulated accounts of the test network. For use with Swarm mainnet, you would need real BZZ.

```
% swarm-cli stamp buy --yes --verbose --depth 20 --amount 1b
```

The batch creation takes some minutes. Eventually it comes back and shows the Stamp ID.

```
Estimated cost: 0.1048576000000000 xBZZ
Estimated capacity: 599.775 MB
Estimated TTL: 2 days
Type: Immutable
At full capacity, an immutable stamp no longer allows new content uploads.
Stamp ID: e2a17af2c560cfaa7229309f1f5b59f0d635dfab8f5e604b8239b3719b51ad82
```

Copy the batch ID (`Stamp ID`) that is displayed for you – the hex digits in the last line – and put them last in the command line that calls the node.js script (example below). The other parameter is the **Bee URL**: `http://localhost:1633`. That is where the Docker Swarm test network API is found.

RUNNING IT

```
node t12.js http://localhost:1633 e2a17af2c560cfaa7229309f1f5b59f0d635dfab8f5e604b8239b3719b51ad82
```

WHAT YOU SHOULD SEE

TERMINAL

```
pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  › using bee url : ‹http://localhost:1633›
pot:  › using batch id: ‹e2a17af2c560cfaa7229309f1f5b59f0…›
pot:  » created new hybrid swarm network loader
pot:  » initialize new P.O.T. - slot 1
pot:  › new slot 1
pot:  › slot ref: 1 da1437aaf2987d14c14bd9ced0360971…
pot:  » put  hello: P.O.T.
pot:  › →  68656c6c6f000000000000000000000000…: 03502e4f2e542e
pot:  » saving storage
pot:  › ref32: b77d8826af7c1684ae88bb63afdee7e3…
pot:  » get  hello: P.O.T.
pot:  › ←  68656c6c6f000000000000000000000000…: 03502e4f2e542e
hello: P.O.T.
```

And that's it. Now let's read back the stored value.

THE SAVE REFERENCE

Using a network rather than in-memory storage, the values we store persist after the script stops.

To read a KVS back from the storage, we need its **save reference**. But the full save reference was cut off in the log as shown above. We could just set a verbosity level that includes the `pot.NOCUT` flag. But we can also retrieve the reference from `save()` and print it to the terminal like this:

```
ref = await kvs.save()
console.log("save reference: " + ref)
```

part of tutorial/t13.js

Running it again, the full save reference is logged to the terminal:

```
save reference: b77d8826af7c1684ae88bb63afdee7e3b81bcb6d0ee15a4d1aa096452dcb7624
```

Note that it is the same reference as the truncated one we saw in the log before – even though a new KVS had been created and the key-value pair stored and saved anew. But we recreated the exact same KVS as before, containing the same key-value pair: ("`hello`", "`P.O.T`"). This shows how the same content results in the same save reference every time. Because the save reference is actually the root of the proximity order tree (POT), which is a complex but deterministic hash of its content.

LOAD

To retrieve from the KVS, create a file `t14.js`:

```
require("./potjs/lib/pot-node")

; (async () => {
        await pot.ready()
        swarmUrl = process.argv[2]
        batchId = process.argv[3]
        saveRef = process.argv[4]
        kvs = await pot.load(saveRef, swarmUrl, batchId)
        value = await kvs.get("hello")
        console.log("hello:", value)
})()
```

full file tutorial/t14.js

The script is similar to the one we used to write to the KVS. But a third parameter is being read from the command line this time: the save reference.

```
        saveRef = process.argv[4]
```

part of tutorial/t14.js

And `new` and the KVS constructor call are replaced by `load()`, which takes one additional parameter, the save reference.

```
kvs = await pot.load(saveRef, swarmUrl, batchId)
```

part of tutorial/t14.js

The `put()` and `save()` calls are deleted. If we can obtain a value for the key `hello` this time around, it will be from the network, i.e., what we stored previously.

RUNNING IT

The parameters are in order: the Docker Swarm client URL (the Bee), the postage batch ID (yours will be a different one) and the **save reference** (yours will be the same one).

```
node t14.js http://localhost:1633 \
e2a17af2c560cfaa7229309f1f5b59f0d635dfab8f5e604b8239b3719b51ad82 \
b77d8826af7c1684ae88bb63afdee7e3b81bcb6d0ee15a4d1aa096452dcb7624
```

WHAT YOU SHOULD SEE

```
pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  » load P.O.T. by reference
pot:  › reference: b77d8826af7c1684ae88bb63afdee7e3…
pot:  › using bee url : ‹http://localhost:1633›
pot:  › using batch id: ‹e2a17af2c560cfaa7229309f1f5b59f0…›
pot:  » created new hybrid swarm network loader
pot:  › load slot 1
pot:  › slot ref: [55b90441c19810e930fc9dc5d739f620…] 1 8432d28d56d2101c195d63a3be907157…
pot:  » get  hello: P.O.T.
pot:  › ←  68656c6c6f000000000000000000000000…: 03502e4f2e542e
hello: P.O.T.
```

The value `P.O.T.` is found for the key `hello`, loaded from the local Swarm network.

Note that the save reference is **not** a handle for the key value store in its different states. It is a handle for a specific state, i.e. only for a KVS populated with the key-values exactly as when it was saved. When it is changed and saved again, a different save reference would return. Because it is the root of a state trie. This makes it possible to store mutable data on an immutable storage network. The save reference also is not a magic seed from which all data can be restored. The retrieval of keys is, of course, from the network. And also the keys are all stored there.

But if the same sequence of updates is run, it will result in the same save reference again when saved.

◊

This concludes the tutorial. Check out **Further Reading** (pg. 21)

FUNCTION OVERVIEW

The main functions are split two ways: **asynchronous** / **synchronous** and **typed** / **raw** respectively. In most cases, only the **asynchronous**, **typed** calls are relevant. Separately, there are initialization, test and support functions.

The central functions are `put()` and `get()`. `get()` returns a promise for a value to be retrieved. And that value will automatically be of the type it was stored in by a `put()` call.

Asynchronous functions return promises and throw an error on failure.

Synchronous functions block and return a value, an error or null. They don't throw errors. The synchronous functions' names end on `Sync`. Synchronous functions can be handy for prototyping and special situations but in the browser only work for tests with the in-memory persistence. From they browser, they don't work when connecting to Swarm or a test network.[15] Therefore, they should not be used in production in the browser. The asynchronous functions work in all circumstances.

Typed functions internally mark the Javascript type of a value, so as to automatically return the right type on retrieval. This can be a `boolean`, a `number`,[16] a `string`, a `Uint8Array`, or `null`.

Raw functions store the byte pattern of any type as it comes. The right function (`getRaw`, `getBoolean`, `getString`, `getNumber`) has to be called later to get the expected type back out. The choice of the function determines the type. The raw functions have `Raw` or a type as name component. Raw functions are useful for special and advanced use cases, and whenever binary data is handled. But the typed, asynchronous functions `put`, `get`, `delete`, etc. will be the first choice.

For more details than given below, and examples, see the **function reference** (separate document).

KVS INITIALIZATION

`pot.Kvs`	constructor to create a new KVS with.
`pot.load`	create a new KVS from a reference of a prior save, async.
`kvs.save`	obtain the root hash of a KVS for retrieval, async.
`pot.loadSync`	create a new Swarm KVS from a reference of a prior save, sync.
`kvs.saveSync`	obtain the root hash of a KVS for retrieval, sync.

TYPE-AWARE

`kvs.put`	return a promise for null or Error. Asynchronous.
`kvs.get`	return a promise for a value, the type will be as had been put. Async.
`kvs.delete`	delete a key-value pair from the KVS. Async.
`kvs.putSync`	return null or Error. Synchronous.
`kvs.getSync`	return a value, the type will be as had been put. Sync.
`kvs.deleteSync`	delete a key-value pair from the KVS. Sync.

[15] It is possible that they might work with the background threads of Web Workers but this has not been tested.

[16] Javascript does not have native integers and stores all numbers as IEEE 754 64-bit floats internally.

RAW BYTE ACCESS

kvs.putRaw	put a value as raw bytes, asynchronous.
kvs.getRaw	get a value as raw Uint8Bytes array, async.
kvs.getBoolean	get a raw byte as Boolean. First byte 0 for false, else true. Async.
kvs.getNumber	get a raw 8-byte value as floating-point number, async.
kvs.getString	get the raw value as string, async.
kvs.putRawSync	put a value as raw bytes, synchronous.
kvs.getRawSync	get a value as raw Uint8Bytes array, sync.
kvs.getBooleanSync	get a raw byte as Boolean. First byte 0 for false, else true, sync.
kvs.getNumberSync	get a raw 8-byte value as floating-point number. Sync.
kvs.getStringSync	get a raw value as string, sync.

POT INITIALIZATION

pot.ready	promise that resolves when pot.wasm is ready to be used.
globalThis.onPotInitialized	called by pot.wasm when it is ready to be used.
pot.setOptimization	controls whether cross-language resources are recycled.
pot.getOptimization	learn whether cross-language resources are being recycled.
globalThis.potOptimization	global variable for setting optimization before initialization
pot.setVerbosity	controls the extent to which POT JS logs, a value from 0 to 5.
pot.getVerbosity	learn the extent to which POT JS logs, a value from 0 to 5.
globalThis.potVerbosity	global variable for setting verbosity before initialization
pot.setValueSizeLimit	soft cap of value byte size. Can be set to any number.
pot.getValueSizeLimit	learn the soft cap of value byte size. Can be set to any number.
pot.byteSize	the size in bytes (not characters) for a Javascript UTF-16 string.
pot.truncString	cut a Javascript UTF-16 string to the specified byte-size.

TESTING AND SUPPORT

These functions are mostly not used in production, and some are no-ops in the production build.

pot.log	write to browser console via Go. Verifies language-land roundtrip
pot.hello	write hello to browser console to verify established WASM stream
pot.randKey	32 random bytes Uint8Array for key testing
pot.randValue	79 to 101 random byte Uint8Array for value testing
pot.randBuffer	arbitrary amount of random bytes for value testing
pot.typeEncodedBytes	Javascript-value encoding with leading type-byte
pot.typeDecodedValue	Javascript type-aware value decoding by leading type-byte
pot.gc	trigger Go garbage collection
pot.prune	compress the Go state map of KVSs
pot.purge	delete the entire in-memory store of key-value pairs
pot.getJSHeapSize	current byte size of the Javascript heap
pot.getGoHeapSize	current byte size of the Go heap
pot.profile	string with Javascript and Go heap size and a bar chart for each
pot.bar	a miniature bar chart for memory tests
pot.hangingPromise	blocking promise with time out for exception testing
pot.panickingPromise	promise that panics internally to test intra-promise recovery
pot.setFail	trigger the next mock storage function called to return an error
pot.setPanic	trigger the next mock storage function called to raise a panic
pot.setDelay	trigger the next mock storage function called to stall for a given time
pot.setHang	trigger the next mock storage function called to stall indefinitely
kvs.release	delete the Go state for a KVS, its Go channels and JS functions

EXAMPLES

This chapter lists and explains ten POT JS code examples.

For a slower, more guided intro, see the **Tutorial** (pg. 22).

RUNNING EXAMPLES

To run html examples (examples 1-6), serve the main project directory and open the example file in the browser, e.g.:

```
npx http-server -o examples/example1.html .
```

or use

```
make example1
```

Do this from `potjs/`, not from the `examples/` folder.

Examples 5, 6, and 9 are started with a number of parameters. It makes sense to use `make` for them. Where not, study the make file rules to learn about the parameters (search "`example5:`" in the `Makefile`, etc.).

THE EXAMPLES

There are eleven examples, 1-11:

`example1.html`	**minimal** in-browser example as above, in-memory storage
`example2.html`	**interactive** in-browser example with input fields, integrity
`example3.html`	like example 2 but without using pot-web.js, plus, delete and save
`example4.html`	like example 1 but using **synchronous** access functions
`example5.html`	like example 1 but with local **Swarm** network storage
`example6.html`	like example 5, plus, save and load, WASM path, and verbosity
`example7.js`	like example 1 but for **node.js**
`example8.js`	similar to example 2, interactive web app for node.js, in-mem.
`example9.js`	identical to example 8, called to use a local Swarm network
`example10.js`	simulated demonstration of cancelation
`example11.js`	ways of error handling

EXAMPLE 1: MINIMAL

A simple put and get, logging to the browser console.

See the previous page on how to run it (`npx-server` or `make`).

- in-browser
- in-memory (as opposed to to-network)
- asynchronous

`examples/example1.html`

```
<script src="lib/pot-web.js"></script>
<script>
(async () => {
      await pot.ready()
      kvs = new pot.Kvs()
      await kvs.put("hello", "POT!")
      value = await kvs.get("hello")
      console.log("key ‹hello› has:", value)
})()
</script>
```

core code of examples/example1.html

WHAT YOU SHOULD SEE

BROWSER

An empty page (but if you used `example1.html`, it has additional text and prints "POT JS Example 1: Hello – Check source and console.") The relevant information is in the browser console:

BROWSER CONSOLE

Open the browser's Javascript console:

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  › created new in-memory persister
pot:  » new slot 1
pot:  › slot ref: 1 2658cee07ea8fce33f29c462f1db1297…
pot:  » put  hello: POT!
pot:  › →  68656c6c6f00000000000000000000000…: 03504f5421
pot:  » get  hello: POT!
pot:  › ←  68656c6c6f00000000000000000000000…: 03504f5421
key ‹hello› has: POT!
```

EXAMPLE 2: INTERACTION

This example shows verbosity setting and integrity checks.

A html form accepts input and shows results. The interaction is entirely in the browser. The sample KVS is stored in an in-memory storage.

The `script` tag in this example has three additional attributes. The first one, `wasm`, is the path to the `pot.wasm` file. It is purely for demonstration in this case as `pot.wasm` is looked for by default in the same folder as `pot-web.js`. In other words, this example does not really look for `pot.wasm` in a different place to not make things complicated.

The example also sets a logging verbosity level, using the `verbosity` attribute. See pg.18 for the list of valid arguments. In the instance it is set to level 3, which is INFO level, producing fewer log lines than the previous example.

The `integrity` attribute uses a hash of the `pot-web.js` script to verify that the correct file arrives in the browser. The example script `examples/example2.html` has the right hash set. The one listed right below might be outdated.

- in-browser
- in-memory
- asynchronous

`examples/example2.html`

```
<pre>

    POT JS Example 2: Interactive, Verbosity, Path, Integrity

    key    <input id=key>
    value  <input id=value>
           <button onclick="put()"> put </button>

<hr />

    key    <input id=search>
           <button onclick="get()"> get </button>
    value  <input id=result>

    <img src=favicon.ico>

</pre>
```

```
<script src="lib/pot-web.js" wasm="lib/pot.wasm" verbosity="3" integrity="sha384-
4hEVHN8uJHWpa4gAFHswdsKiz5Ss9sFjp/X+HLrZGsqVsQunkRmMdFojIZ38s64T"></script>

<script>

    var kvs

    (async () => {
            await pot.ready()
            kvs = new pot.Kvs()
    })()

    const field = (id) => { return document.getElementById(id) }

    async function put() {
            key = field('key').value
            value = field('value').value
            await kvs.put(key, value)
    }

    async function get() {
            key = field('search').value
            value = await kvs.get(key)
            field('result').value = value
    }

</script>
```

examples/example2.html

RUNNING IT

```
npx http-server -o examples/example2.html .
```

or use

```
make example2
```

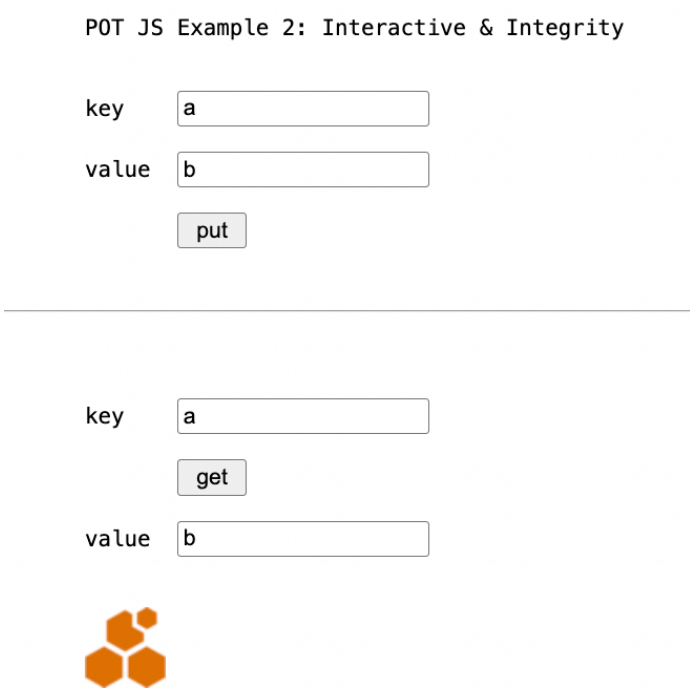
1) Enter a value for **key** and **value** and click **put**.

2) Enter the same **key** again in the lower screen area and click **get**.

This will yield the **value** in the lowest field.

WHAT YOU SHOULD SEE

BROWSER



screen of example 2

BROWSER CONSOLE

Because verbosity was set to 3 in the script attribute, there are fewer details shown than in example 1. The binary key and value hex digits are left out, as are all lines starting with ›.

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  » created new in-memory persister
pot:  » new slot 1
pot:  » put  a: b
pot:  » get  a: b
```

EXAMPLE 3: ROOT, DELETE, ALTERNATE INITIALIZATION

Demonstration of trie root and deletion.

This example has a more **explicit initialization**, leaving out the Javascript wrapper: it does not use `pot-web.js` but initializes `pot.wasm` directly.[17] This can be helpful to gain more control but works only in the browser. Instead of the script tag attribute `verbosity`, this example uses `globalThis.potVerbosity` to set a log level.

To be more diligent, this example should have a mechanism – using `pot.ready()` or `globalThis.onPotInitialized()` (see example 4, pg. 58) – that would prevent its `get()`, `put()`, `delete()` and `save()` functions to be called before WASM initialization is complete. In practice, the buttons in this form will not be pressed fast enough by a user to race WASM initialization. However, cleaner code might just add `pot.ready()` everywhere before `put()` etc.

- in-browser
- in-memory
- asynchronous

`examples/example3.html`

```
<pre>

  POT JS Example 3: The Root, Alternate Initialization, Delete

  This page demonstrates the trie root value and to that end,
  how to delete key-value pairs from the KVS. It does not use
  pot-web.js but initializes pot.wasm directly. And it uses a
  global variable to set verbosity.

  key    <input id=key onfocus="field('value').value = ''">
  value  <input id=value>
         <button onclick="put()"> put </button>
         <button onclick="get()"> get </button>
         <button onclick="del()"> delete </button>
  root   <input id=root style=width:500>

  <img src=favicon.ico>

</pre>
```

[17] Note that the corollary is not possible for node.js with network storage because that needs a Javascript fetch function out of `pot-node.js`.

```
<script src="lib/wasm_exec.js"></script>
<script>

 var kvs
 const go = new Go()
 window.potVerbosity = 3
 WebAssembly.instantiateStreaming(fetch("lib/pot.wasm"), go.importObject)
     .then(async (r)=>{
            go.run(r.instance)
            kvs = new pot.Kvs()
     })

 const field = (id) => { return document.getElementById(id) }

 async function put() {
     key = field('key').value
     value = field('value').value
     await kvs.put(key, value)
     save() // note this is not kvs.save()
 }

 async function get() {
     key = field('key').value
     value = await kvs.get(key)
     field('value').value = value
 }

 async function del() {
     key = field('key').value
     await kvs.delete(key)
     field('value').value = ''
     save()
 }

 async function save() {
     ref = await kvs.save(key)
     field('root').value = ref
     pot.log("root: " + ref)
 }
</script>
```

Full source of examples/example3.html

RUNNING IT

```
npx http-server -o examples/example3.html .
```

or use

```
make example3
```

The example demonstrates how the root value of the proximity order trie (POT) works.

1. Store key `foo`, value `bar`
2. Store `bar baz`
3. Store `bar quz`
4. Delete `bar`

At each step, the last field on-screen, the root hash display, will change. Relevantly, after step 4 it will display the same value again as after step 1. And it's predictable from the key-value pairs put:

`a0b364aed38efa98a8f8bcca8b3a10181c23e6535c2c413b79a5ff662852d161`.

This illustrates neatly how the save reference (see pg. 9), which *is* the POT root, *is* the hash of the trie of values in the KVS. It changes with the key-values of a KVS changing, and goes back to a previous value when the key-values of a KVS are the same as before again. The way the values are hashed for the Swarm POT implementation is involved, because it encompasses the POT magic of how the trie is built.[18] But the principle demonstrated is that the save reference is really the expression of a specific state – as opposed to a specific object that changes state. Also note that every time you run the script, it will show the same root hashes.

This is what bridges from the world of mutable program state to that of immutable storage.

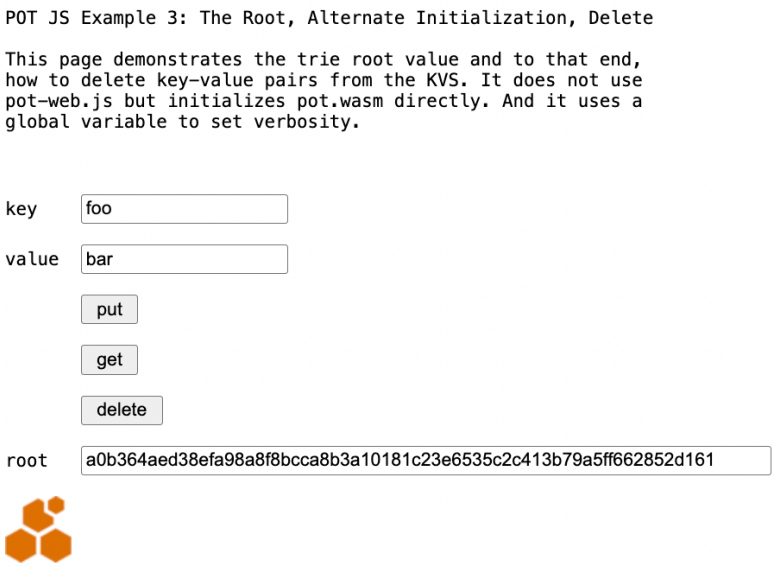
The `save()` call is otherwise useless in *this* scenario, because we are using the transient in-memory persister. But it triggers the logging of the current POT root to the browser console.

This example is also different from the previous example, in that it does not use the Javascript wrapper `pot-web.js` but initializes `pot.wasm` on its own and explicitly includes the generic `wasm_exec.js` instead that is the bare bones Javascript wrap around any Go WASM. The variant of this for node.js is shown in example 8 (pg. 65).

[18] Cf. the forthcoming paper on POT and the Book of Swarm for the details about Swarm chunk hashing. The reason it can get involved is that large values are compartmentalized into multiple subsections, the chunks.

WHAT YOU SHOULD SEE

BROWSER



Example 3 after the first step, storing **foo: bar**.

BROWSER CONSOLE

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  » created new in-memory persister
pot:  » new slot 1
pot:  » put  foo: bar
pot:  » saving storage
pot:  root: a0b364aed38efa98a8f8bcca8b3a1018…
pot:  » put  bar: baz
pot:  » saving storage
pot:  root: 707ccce2f13337d9d98e7d49d8255e82…
pot:  » put  bar: qux
pot:  » saving storage
pot:  root: 5a73b03b8be0c21d75061a27519ca6dd…
pot:  » delete  bar
pot:  » saving storage
pot:  root: a0b364aed38efa98a8f8bcca8b3a1018…
```

EXAMPLE 4: SYNCHRONOUS

Otherwise like example 1 but using synchronous calls.

Note that sync calls don't work connected to a network in the browser. They just return an error. That's not a terrible loss because the browser should really not be frozen with sync calls to a network. They work fine in every circumstance with node.js, and with every setup that uses in-memory storage.

- in-browser
- in-memory
- synchronous

examples/example4.html

```
...
<script src="lib/pot-web.js"></script>
<script>
function onPotInitialized() {
    kvs = new pot.Kvs()
    kvs.putSync("hello", "POT!")
    value = kvs.getSync("hello")
    console.log("key ‹hello› has:", value)
}
</script>
```

core of examples/example4.html

WHAT YOU SHOULD SEE

Almost the same as before in example 1.

BROWSER

```
POT JS Example 4: Synchronous

Check source and console.

This page uses synchronous calls that can be helpful for prototyping
but don't work for network connections (only in-memory, i.e using
newSync() without parameters) and not for use with node. Wrong use
will just reliably deadlock.
```

BROWSER CONSOLE

The same as for example 1, except the slot hash will differ.

EXAMPLE 5: NETWORK

Connect to a local Swarm network instead of using in-memory storage.

This is mostly the same code as example 1, it just supplies network parameters to `new`.

The code listed below is for illustration: the `batch id` will have to be updated to run it. But `make` does that for you when you use `make example5`. It patches the source code of `example/example5.html` with a new batch id. The one listed below will not be the one.

- in-browser
- network
- asynchronous

`examples/example5.html`

```
<pre>

    POT JS Example 5: Network
    Check source and browser console.

    This page connects to a local Swarm network.

    It persists a value, and does not read it back. put() stores to
    cache, save() stores to the network.

    To test it, use `make example5`, which will start a local Swarm
    network and update the batch id literal in the pot.new() call
    below after creating a new batch. Note that some other make rules
    will re-use this batch id, stored in .batch_id, if it exists.

    <img src=favicon.ico>

</pre>

<script src="lib/pot-web.js"></script>
<script>

(async () => {
    await pot.ready()
    kvs = new pot.Kvs("http://localhost:1633",
"12c073a8b2a8f94b3013d1794e6ad713905d4083503b2180a798e2d5841c6666")
    await kvs.put("hello", "POT!")
    ref = await kvs.save()
    console.log("tree root is:", ref)
})()

</script>
```

examples/example5.html

RUNNING IT

This example is a little more involved, it needs a docker engine and e.g., the Fair Data Society's Swarm test network[19] installed, as well as the Swarm CLI. A postage batch ID (see pg. 15) needs to be created, and as noted above, once obtained, the id must be patched into examples/example5.html. This is all done by the make rules.

Install docker, and do

```
% make locnet_install example5
```

The creation of the postage batch takes some minutes. For more details on it see the instructions, (pg. 15) and tutorial (pg. 43). For clarity, `locnet_install` calls:

```
npm install --global @ethersphere/swarm-cli
npm install --global @fairdatasociety/fdp-play
```

WHAT YOU SHOULD SEE

TERMINAL

```
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
⬢ √ pot.wasm built for production
server still running
/Library/Developer/CommandLineTools/usr/bin/make locnet_start
⬡ starting five local swarm nodes
on failure despite docker running, try erasing existing fdp-play containers
fdp-play start --detach
All containers are up and running
rm -f .batch_id
/Library/Developer/CommandLineTools/usr/bin/make .batch_id
⬡ buy stamps (free in this test setup)
swarm-cli stamp buy --yes --verbose --depth 20 --amount 1b | tee .batch_creation
Estimated cost: 0.1048576000000000 xBZZ
Estimated capacity: 599.775 MB
Estimated TTL: 2 days
Type: Immutable
At full capacity, an immutable stamp no longer allows new content uploads.
Stamp ID: 37004b73068c18e024d8221ff55b65847ba7d63b739ff44785ec233d0163dced
grep "Stamp ID:" .batch_creation | cut -c11-74 > .batch_id
⬡ postage batch ID: 37004b73068c18e024d8221ff55b65847ba7d63b739ff44785ec233d0163dced
sed -i.bak -e "s/\(pot\.Kvs.*\)\"[0-9a-fA-F]*\")/\1\"$(cat .batch_id)\")/"
examples/example5.html ; rm -f examples/example5.html.bak
open http://127.0.0.1:8080/examples/example5.html
% ...  "GET /examples/example5.html" ...
...  "GET /examples/lib/pot-web.js" ...
...  "GET /examples/favicon.ico" ...
...  "GET /examples/lib/pot.wasm" ...
```

[19] https://github.com/fairDataSociety/fdp-play – Discord https://discord.gg/KrVTmahcUA

BROWSER CONSOLE

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:  › using bee url : ‹http://localhost:1633›
pot:  › using batch id: ‹37004b73068c18e024d8221ff55b6584…›
pot:  » created new generic swarm network loader
pot:  » initialize new P.O.T. - slot 1
pot:  › slot ref: 1
pot:  » put  hello: POT!
pot:  › ⟶  68656c6c6f0000000000000000000000…: 03504f5421
pot:  » saving storage
pot:  › ref32: 5968b03183efcb859b135c21e5e98b37…
tree root is: 5968b03183efcb859b135c21e5e98b378e189789cfa3593761a74b69fca0aed3
```

As noted, your batch ID will differ.

EXAMPLE 6: NETWORK II

Saving and re-loading a KVS.

This example demonstrates how a KVS is persisted and how a new KVS object is instantiated for it, using `save()` and `load()`. Different from the previous example, it receives the batch ID as parameter. But note that it re-uses the network setup and the batch ID that we created for example 5.

Regarding the meaning of the save reference, see pg. 9. In a nutshell, `kvs1` and `kvs2` in the example code below are not two different key-value-stores – but also not the same. Because the save reference means a certain *state*, not a certain store's identity. Every store that has the same contents will have the same save reference. But every KVS object can have its own state. Sometimes, it will be the same state like another KVS object. But that does not mean the two KVS objects are now the same.

- in-browser
- network
- asynchronous

`examples/example6.html`

```
<pre>
     POT JS Example 6: Network II
     Check source and browser console.

     This page connects to a local Swarm network.
     It can be tried by running `make example6`, which takes care
```

```
    of the local network and the batch id. The script connects to
    the http server at port 8080, the Swarm queen node at 1633.

    What those make rules do is essentially:

      fdp-play start --detach
      swarm-cli stamp buy --yes --verbose --depth 20 --amount 1b > .batch_creation
      grep "Stamp ID:" .batch_creation | cut -c11-74 > .batch_id
      npx http-server -c1 . &
      open "http://127.0.0.1:8080/example6.html?bee=http://127.0.0.1:1633&batch=$(cat .batch_id)"

    If the batch id file exists, or the http server is already running, the
    make rules will skip the respective lines.

    Use `make stop` to stop the servers, or keep them running for the next
    examples.

    <img src=favicon.ico>

</pre>

<script src="lib/pot-web.js" verbosity=2></script>
<script>

(async () => {

    await pot.ready()

    batch = new URLSearchParams(window.location.search).get('batch');

    kvs1 = await pot.new("http://localhost:1633", batch)
    console.log("kvs #1 -- put hello      : POT!")
    await kvs1.put("hello", "POT!")
    value = await kvs1.get("hello")
    console.log("kvs #1 -- key ‹hello› has:", value)
    ref1 = await kvs1.save()
    console.log("kvs #1 -- trie root is   :", ref1)

    kvs2 = await pot.load(ref1, "http://localhost:1633", batch)
    value2 = await kvs2.get("hello")
    console.log("kvs #2 -- key ‹hello› has:", value2)
    ref2 = await kvs2.save()
    console.log("kvs #2 -- trie root is   :", ref2)

    console.log("kvs #2 -- put baz        : true")
    await kvs2.put("baz", true)
    ref3 = await kvs2.save()
    console.log("kvs #2 -- trie root is   :", ref3)

})()
</script>
```

examples/example6.html

RUNNING IT

This example uses the setup of example 5 – the local network, the postage batch etc.

Run example 5 to prepare for this. Then do

```
make example6
```

WHAT YOU SHOULD SEE

TERMINAL

```
% make example6
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
⬢ √ pot.wasm built for production
server still running
/Library/Developer/CommandLineTools/usr/bin/make locnet_start
⬡ starting five local swarm nodes
on failure despite docker running, try erasing existing fdp-play containers
fdp-play start --detach
All containers are up and running
/Library/Developer/CommandLineTools/usr/bin/make .batch_id
make[1]: `.batch_id' is up to date.
open "http://127.0.0.1:8080/examples/example6.html?bee=http://127.0.0.1:1633&batch=$(cat
.batch_id)"
... % ... "GET /examples/example6.html?bee=http://127.0.0.1:1633&batch=37004b73068c1...
... "GET /examples/favicon.ico" ...
... "GET /examples/lib/pot-web.js" ...
... "GET /examples/lib/pot.wasm" ...
```

BROWSER

Some explanations.

BROWSER CONSOLE

Verbosity is this time set to 2, which effects that beyond the `console.log()` messages from the example code, only error messages would be shown from the POT JS engine. As a result you should only find the explicit console.log() calls in the source above showing up in the console.

```
kvs #1 -- put hello      : POT!
kvs #1 -- key ‹hello› has: POT!
kvs #1 -- trie root is   : 5968b03183efcb859b135c21e5e98b378e189789cfa3593761a74b69fca0aed3
kvs #2 -- key ‹hello› has: POT!
kvs #2 -- trie root is   : 5968b03183efcb859b135c21e5e98b378e189789cfa3593761a74b69fca0aed3
kvs #2 -- put baz        : true
kvs #2 -- trie root is   : 92bc2a4cf5b67780bfa289ddf32b558c11cf88d53d8c793171d97bc2442a22dc
```

The trie roots you see should be the very same as above.

EXAMPLE 7: NODE

node.js instead of in-browser.

This example shows how POT JS works the same with node.js as in the browser.

pot-node.js is required instead of **pot-web.js**. The former is a little heavier than the latter because it contains the implementation of a blocking fetch that is required by Go POT to work with node.js. The reason being that WASM inherits the browser's fetch for free but there is no equivalent of that for WASM running with node.js. Rather, no network interaction is possible for a WASM executable running with node.js. That's why **pot-node.js** provides it.

- node.js
- in-memory
- asynchronous

examples/example7.js

```
console.log(`
    POT JS Example 7: node.js
    Check source and terminal.
`)
require("./lib/pot-node")
; (async () => {
    await pot.ready()
    kvs = new pot.Kvs()
    await kvs.put("hello", "node")
    value = await kvs.get("hello")
    console.log("hello:", value)
})()
```

examples/example7.html

RUNNING IT

This example is plain and can be run without any further preparation by

```
node examples/example7.js
```

or

```
make example7
```

WHAT YOU SHOULD SEE

Essentially the same as in example 1, just on the terminal instead of in the browser console.

TERMINAL

```
% node examples/example7.js

     POT JS Example 7: node.js
     Check source and terminal.

pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  » created new in-memory persister
pot:  » new slot 1
pot:  › slot ref: 1 5f79e6247c8e438966f01a09b3e6ada9…
pot:  » put  hello: node
pot:  › →  68656c6c6f00000000000000000000000…: 036e6f6465
pot:  » get  hello: node
pot:  › ←  68656c6c6f00000000000000000000000…: 036e6f6465
hello: node
```

The slot reference will vary but all other numbers should be the same.

EXAMPLE 8: NODE WEB APP

A stub node.js web server.

This example is a miniature web-app that does the same as example 2, but this time, the logic runs on the server side, node.js, instead of in the browser. The result is very similar, so is the source code. The page is served hardcoded from the script itself. The POT functions run on the server, not in the browser.

The top part of the source code is the HTML that is sent to the browser. Then follow the initialization of POT and the Javascript `get()` and `put()` function that the buttons in the web form will call. This time not via Javascript in the browser but by HTML POSTs to the server, where the `get()` and `put()` functions reside now. This may deserve repeating because it is so similar to example 2. But they run at a different time, connected to the web page in a different way. The last part is the web server.

The script also demonstrates the use of `WebAssembly.instantiate()` for node.js, leaving `pot-node.js` out. In involved scenarios the higher control might be useful. Note that this will not work for node.js with network access because for the network case, `potnode.js` provides additional required code, the blocking `jsFetch()` function and thus `pot-node.js` cannot be left out.

- node.js
- in-memory
- asynchronous

examples/example8.js

```
console.log(`

    POT JS Example 8: Node.js, in-memory, explict init, asynchronous

    This is a self-contained web app, serving a page to receive input,
    returning the results.

    open http://localhost:3000
`)

const http = require("http")
const qs = require("querystring")
const fs = require("fs")
require("./lib/wasm_exec")

const form = `<pre>

            POT JS Example 8: Node.js Web App Server

            <form method=post action="http://localhost:3000">

            key    <input name=key>
            value  <input name=value>
                   <input type=submit name=button value=put>
    <hr />
            key    <input name=search>
                   <input type=submit name=button value=get>
            value  <input name=result>

            </form>

    <hr />

    </pre>

    <script> console.log('see server log in terminal') </script>

    <pre>
            `

var kvs

var go = new Go()

globalThis.potVerbosity = 3

WebAssembly.instantiate(fs.readFileSync("lib/pot.wasm"), go.importObject)
    .then((r) => { go.run(r.instance) })

global.onPotInitialized = async () => {

    bee = process.argv[2]
    batch = process.argv[3]
    kvs = new pot.Kvs(bee, batch)

    // There is no catching of early calls of put() and get()
    // in this example, which in theory could race the loading
    // of pot.wasm and the creation of kvs.
```

```
}

async function put(key, value) {
    await kvs.put(key, value)
    return `put ${key}: ${value}`
}

async function get(search) {
    value = await kvs.get(search)
    return `get ${search}: ${value}

            <script>
            document.getElementsByName('result')[0].value = '${value}'
            </script>`
}

const server = http.createServer((request, response) => {

    var log  = ''
    var body = ''

    request.on('data', (data) => body += data)
            .on('end', async () => {
                    const { button, key, value, search } = qs.parse(body)
                    switch(button) {
                    case 'put': log = await put(key, value) ; break
                    case 'get': log = await get(search)
                    }
                    response.writeHead(200)
                    response.end(form + log + `\n</pre>`)
    })
})
.listen(3000, '127.0.0.1')

console.log('srv: listening at http://127.0.0.1:3000')
```

examples/example8.js

RUNNING IT

This example can be run without any further preparation by

```
node examples/example8.js
```

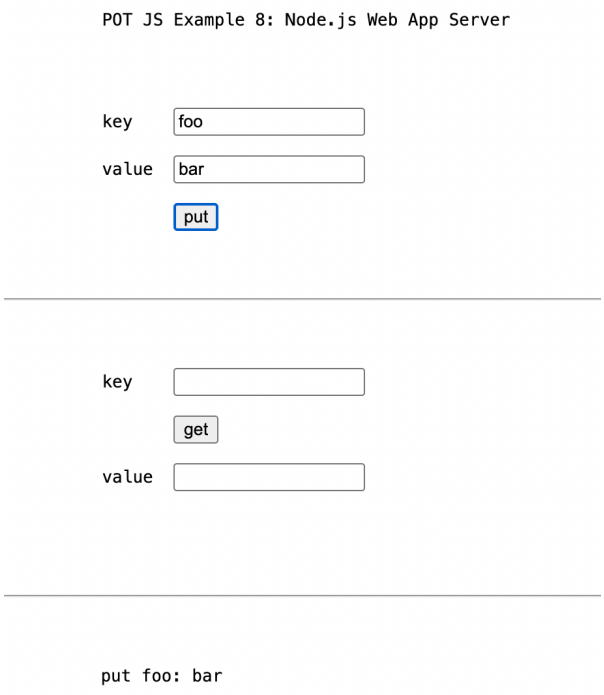
or

```
make example8
```

WHAT YOU SHOULD SEE

BROWSER

The screen will refresh with fields empty on every button press.



montage of example 8 before and after pressing put

TERMINAL

```
% make example8
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
⬢ √ pot.wasm built for production
open http://127.0.0.1:3000
node examples/example8.js

     POT JS Example 8: Node.js, in-memory, explict init, asynchronous

     This is a self-contained web app, serving a page to receive input,
     returning the results.

     open http://localhost:3000

srv: listening at http://127.0.0.1:3000
pot:  verbosity set 3
pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  » created new in-memory persister
pot:  » new slot 1
pot:  » put  foo: bar
pot:  » get  foo: bar
```

EXAMPLE 9: NODE APP / NETWORK III

node.js Web-app with real storage.

Examples 8 and 9 are at their core identical to make the point that the storage mode and destination for a KVS can be switched by parameter. No change in the source code is needed.

But example 9 also shows how the **async** calls of example 8 can instead be written **synchronously**. The result is that functions like `get()` are replaced by `getSync()`, etc. the `await` goes away of course, and the containing function can drop the `async` tag. A special caveat is that `*Sync()` functions do not throw errors but return them as result. But error handling is not demonstrated.

Verbosity is set as an extra parameter to `require` (in its own brackets). And the normal way of doing things (where `pot-node.js` is included) is shown, instead of using `WebAssembly.instantiate()`.

This example uses the test network, and re-uses the batch ID, of example 5. See pg. 59. You should run example 5 to get it prepared.

- node.js
- local network
- synchronous

examples/example9.js

```
...

require("./lib/pot-node")(3)

...

var kvs

global.onPotInitialized = () => {
    bee = process.argv[2]
    batch = process.argv[3]
    kvs = new pot.Kvs(bee, batch)
}

function put(key, value) {
    kvs.putSync(key, value)
    return `put ${key}: ${value}`
}

function get(search) {
    value = kvs.getSync(search)
    return `get ${search}: ${value}

            <script>
            document.getElementsByName('result')[0].value = '${value}'
            </script>`
}

const server = http.createServer((request, response) => {

    var log  = ''
    var body = ''

    request.on('data', (data) => body += data)
            .on('end', () => {
                    const { button, key, value, search } = qs.parse(body)
                    switch(button) {
                    case 'put': log = put(key, value) ; break
                    case 'get': log = get(search)
```

```
                    }
                    response.writeHead(200)
                    response.end(form + log)
    })
})
.listen(3000, '127.0.0.1')

console.log('srv: listening at http://127.0.0.1:3000')
```

examples/examples9.js

RUNNING IT

This example uses the setup of example 5 – the local network, the postage batch etc. Set up example 5 to prepare for this. Then do

```
make example9
```

WHAT YOU SHOULD SEE

BROWSER

Like example 8.

TERMINAL

Basically, the same as for example 8.

```
% make example9
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
⬢ √ pot.wasm built for production
/Library/Developer/CommandLineTools/usr/bin/make locnet_start
⬡ starting five local swarm nodes
on failure despite docker running, try erasing existing fdp-play containers
fdp-play start --detach
All containers are up and running
/Library/Developer/CommandLineTools/usr/bin/make .batch_id
make[1]: `.batch_id' is up to date.
open http://127.0.0.1:3000
node examples/example9.js "http://127.0.0.1:1633" "$(cat .batch_id)"

     POT JS Example 9: Node.js, local Swarm network, synchronous

     This is a self-contained web app, serving a page to receive input,
     returning the results.

     Functionally identical to example 8, just: started with different
     arguments – a network url and a batch id – by the calling make rule;
     and POT calls used are synchronously blocking instead of promises.

     open http://localhost:3000

srv: listening at http://127.0.0.1:3000
pot:  » POTWASM
pot:  » node.js detected
pot:  » init done
pot:  » ready
pot:  » created new hybrid swarm network loader
pot:  » new slot 1
pot:  » put  foo: bar
pot:  » get  foo: bar
```

EXAMPLE 10: CANCELATION

Canceling transactions.

Example 10 uses a specialized build of the WASM file, for the simulation of exceptions.

The example uses the special test function `pot.setDelay()` to make the function that follows it delay its execution. This makes it possible to test the `cancel()` method of the `put()`. Check out pg. 13 for details on how to use `cancel()`. Unless `pot.wasm` is built for simulation, `pot.setDelay()` is a no-op.

- node.js
- simulation
- asynchronous

examples/example10.js

```
console.log(`
    POT JS Example 10: cancel
    Check source and terminal.
`)

require("./lib/pot-node")
; (async () => {
    await pot.ready()
    pot.setVerbosity(pot.INFO)
    kvs = new pot.Kvs()
    pot.setDelay(1000)
    put = kvs.put("hello", "node")
    put.catch((e)=>console.log(">>>> caught in kvs.put", e.message))
    put.cancel()
    console.log(">>>> value of hello:", await kvs.get("hello"))
})()
```

RUNNING IT

This example requires the special 'simulation' build of `pot.wasm`:

```
make mockbuild
node examples/example10.js
```

or

```
make example10
```

WHAT YOU SHOULD SEE

TERMINAL

```
% make example10
go mod edit -dropreplace github.com/brainiac-five/pot
go mod edit -replace github.com/brainiac-five/pot=./mock
go get github.com/brainiac-five/pot/pkg/persister
GOOS=js GOARCH=wasm go build -tags=ext_test -o lib/pot.wasm potjs.go swarm_nodejs.go mock.go
● √ pot.wasm built for extended API tests
node examples/example10.js

     POT JS Example 10: cancel

     Check source and terminal.

pot:  080  » POTWASM
pot:  082  » node.js detected
pot:  083  » init done
pot:  083  » ready
pot:  084  » created new in-memory persister
pot:  085  » new slot 1
pot:  085  » set up delay of 1000 ms
pot:  086  » mock delay …
pot:  088  x put canceled
pot:  088  » mock delay done at 2 ms
pot:  089  ### exception in put*(): context canceled
pot:  089  » get  hello: undefined
>>>> caught in kvs.put: ### exception in put*(): context canceled
>>>> value of hello: undefined
```

`pot.setDelay()` pauses the `put()` call internally for one second (1000 ms). It is a function that exists specifically to create situations not on the happy path to be able to test error handling. This works only with the simulation (mock) build of `pot.wasm`, which is for testing error catching, timeouts, and cancelation. `setDelay()` is a no-op in the normal build of `pot.wasm`: the functions exists but it does nothing. This and other test functions are described in the function reference doc.

In the source code shown above, the `promise` that `put()` returns, rejects when `cancel()` is called on it. The `.catch()` method receives the rejection that contains the cancel message as error parameter.

Because it was canceled, the `put()` did not succeed. The later `get()` call of with the key `"hello"` receives `undefined` as value, showing that the entry does not exist.

Note that the cancel will not happen if this example is run with the normal `pot.wasm` build. Because `setDelay()` will not actually set a delay and then the `put()` will complete faster than the `cancel()` comes into effect.

EXAMPLE 11: ERROR HANDLING

Ways of catching and receiving errors.

Example 11 also uses a specialized build of the WASM file, for the simulation of exceptions.

The example uses the special test functions `pot.setFail()` and `pot.setHang()` to make the KVS function that is called next to fail or hang. This makes it possible to test error handling and the `cancel()` method. Check out pg. 13 for details on how to use `cancel()`. Unless `pot.wasm` is built for simulation, `pot.setFail()` and `pot.setHang()` are a no-op, i.e., they normally have no effect.

- node.js
- simulation
- asynchronous

examples/example11.js

```
console.log(`

      POT JS Example 11: error handling

      Check source and terminal.
`)

require("./lib/pot-node")(1)

; (async () => {

      await pot.ready()
      kvs = new pot.Kvs()

      // .catch
      console.log("      ► chained .catch()")
      pot.setFail(true)
      await kvs.put("hello", "node")
       .catch(e=>console.log("      →   caught in chained .catch: " + e))

      // cancel()
      console.log("      ► cancel")
      pot.setHang(true)
      put = kvs.put("hello", "node") // cancel() works only when .catch() is not
      chained to put() itself
      put.catch(e=>console.log("      →   caught in cancel .catch: " + e))
      put.cancel()

      // timout
      console.log("      ► timeout")
      pot.setHang(true)
      timeout = 10 // ms
      put = kvs.put("hello", "node", timeout)
      put.catch(e=>console.log("      →   caught in timeout .catch: " + e))
```

```
    // try-catch
    console.log("      ► try-catch block")
    pot.setFail(true)
    try {
    await kvs.put("hello", "node")
    } catch(e) {
    console.log("      →   caught in try-catch block: " + e)
    }

    // sync call
    console.log("      ► sync call")
    pot.setFail(true)
    e = kvs.putSync("hello", "node") // sync calls don't throw but return errors
    if(e) {
    console.log("      →   returned from sync call: " + e)
    }
})()
```

RUNNING IT

This example requires the special 'simulation' build of **pot.wasm**:

```
make mockbuild
node examples/example11.js
```

or

```
make example11
```

WHAT YOU SHOULD SEE

TERMINAL

```
% make example11
go mod edit -dropreplace github.com/ethersphere/proximity-order-trie
go mod edit -replace github.com/ethersphere/proximity-order-trie=./mock
go get github.com/ethersphere/proximity-order-trie/pkg/persister
GOOS=js GOARCH=wasm go build -tags=ext_test -o lib/pot.wasm potjs.go swarm_nodejs.go mock.go
⬢ √ pot.wasm built for extended API tests
node examples/example11.js

    POT JS Example 11: error handling

    Check source and terminal.

     ► chained .catch()
     →   caught in chained .catch: Error: ### exception in put*(): mock fail of Put()
     ► cancel
     ► timeout
     ► try-catch block
     →   caught in cancel .catch: Error: ### exception in put*(): context canceled
     →   caught in try-catch block: Error: ### exception in put*(): mock fail of Put()
     ► sync call
     →   returned from sync call: Error: ### exception in put*(): mock fail of Put()
     →   caught in timeout .catch: Error: ### exception in put*(): context deadline exceeded
```

There are five different ways demonstrated how errors can be caught or received. As explained before, the sync functions do not throw errors, but return them.

A major caveat is that **.catch()** must not be chained immediately to **put()** (or any other KVS function). The promise returned from put() must be assigned to a variable first; and to that variable both **.catch()** and **.cancel()** are attached separately. Check out pg. 13 for details.

Upon inspection of the output log, the error catches do not happen strictly sequentially. For example, the .catch of the cancel segment is logged only after the try-catch block has started.

```
► cancel
► timeout
► try-catch block
→   caught in cancel .catch: Error: ### exception in put*(): context canceled
→   caught in try-catch block: Error: ### exception in put*(): mock fail of Put()
...
→   caught in timeout .catch: Error: ### exception in put*(): context deadline exceeded
```

That is because the asynchronous handling of promise executors happens in-between jobs of the main function of the test has completed.

BUILDING

There is no need to build POT JS to use it.

The main executable, the Go WASM file pot.wasm, the Javascript & HTML files are portable.

To build, run

```
% GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
```

or use

```
% make build
```

or

```
% make clean build
```

WHAT YOU SHOULD SEE

```
% make build
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
⬢ √ pot.wasm built for production
```

CLEAN BUILD

To rebuild from scratch, including integrity hashes and rebuilding `go.mod`, use:

```
% make clean build
```

WHAT YOU SHOULD SEE

```
% make clean build
⬡ clean to rebuild from scratch
rm -f go.mod
rm -f lib/pot.wasm
rm -f lib/wasm_exec.js
rm -f lib/potjs.js.sha384 lib/wasm_exec.js.sha384
rm -f .batch_creation .batch_id
go mod init potjs
go: creating new go.mod: module potjs
go: to add module requirements and sums:
     go mod tidy
go mod edit -replace github.com/ethersphere/proximity-order-
trie=github.com/ethersphere/proximity-order-trie@v1.0.2-alpha.7
go get
go: added github.com/beorn7/perks v1.0.1
go: added github.com/cespare/xxhash/v2 v2.2.0
go: added github.com/ethersphere/bee/v2 v2.5.0
go: added github.com/ethersphere/proximity-order-trie v1.0.0
go: added github.com/hashicorp/errwrap v1.0.0
go: added github.com/hashicorp/go-multierror v1.1.1
go: added github.com/prometheus/client_golang v1.18.0
go: added github.com/prometheus/client_model v0.6.0
go: added github.com/prometheus/common v0.47.0
go: added github.com/prometheus/procfs v0.12.0
go: added golang.org/x/crypto v0.23.0
go: added golang.org/x/sys v0.20.0
go: added google.golang.org/protobuf v1.33.0
cp "$(go env GOROOT)/lib/wasm/wasm_exec.js" lib/
openssl dgst -sha384 -binary lib/wasm_exec.js | openssl base64 -A > lib/wasm_exec.js.sha384
GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
⬢ √ pot.wasm built for production
```

REQUIREMENTS

POT JS includes the Go implementation of POT. It does not use Javascript dependencies.

POT JS

Tested with:

- go 1.24.0
- node.js 22.17.1
- chrome 138.0.7204.184 arm64

likely works with earlier versions.

Tests with a local Swarm network include

- @ethersphere/swarm-cli v2.36.1 works
- @fairdatasociety/fdp-play v3.3.0 works

GO POT

- go 1.24.0

- testify 1.8.4
- swarm bee 2.5.0
- proximity-order-trie 1.0.0
- x/crypto 0.23.0

Which will require

- perks 1.0.1
- xxhash 2.2.0
- errwrap 1.0.0
- go-multierror 1.1.1
- prometheus
- x/sys 0.20.0
- protobuf 1.33.0

See `go.mod`.

TESTS

POT JS uses **Jest installation tests** to check system compatibility and a **custom deep test suite** to test cross-environment behavior and memory management.

INSTALLATION TESTS

Installation tests are implemented using Jest. There are 187 tests[20] that check the health of the installation and whether POT JS works on the given platform as a server. They use the in-memory version of POT. They do not test behavior in the browser or network storage.

Run the tests with

```
% make test
```

or

```
% npx jest --config test/jest.json --runInBand --testRegex jest..\*.js$
```

Test suites are in the files

```
test/jest.json
test/jest.async.js
test/jest.sync.js
```

Use the deep tests (see pp. 81) to check network interaction, in-browser operation and be protected against more regressions when contributing to POT JS.

[20] As of version 1.0.10

WHAT YOU SHOULD SEE

TERMINAL

```
% make test
#
#  The jest installation test suite is run by:
#
#    % make test
#
#  Deep regression tests can be run in 2x3x2 combinable deep test modes:
#
#    browser / node + in-memory / simulated / network + standard / stress.
#

...

GOOS=js GOARCH=wasm go build -o lib/pot.wasm potjs.go swarm_nodejs.go nomock.go
● √ pot.wasm built for production

●  Jest Tests

npx jest --config test/jest.json --runInBand --testRegex jest..\*.js$
 PASS  test/jest.util.js
 PASS  test/jest.async.js
 PASS  test/jest.sync.js

Test Suites: 3 passed, 3 total
Tests:       187 passed, 187 total
Snapshots:   0 total
Time:        4.838 s, estimated 5 s
Ran all test suites.
```

TEST VERBOSITY

To increase verbosity, add `--verbose` to the `jest` call. This shows jest's list of all tests.

```
% npx jest --config test/jest.json --verbose --runInBand --testRegex jest..\*.js$
```

This lists the names of the individual tests being run.

```
Synchronous Regression Tests
    ✓ Save after storing identical key-value twice - sync
    ✓ Save after deletion of non-existent key (non-empty KVS) - sync
    ✓ Saving a KVS empty returns 0 reference - sync
    ✓ Create a KVS from a 0 reference - sync (1 ms)
```

To increase it more, edit `test/jest.json` and set `verbosity: true`.

```
{
  "verbose": true
}
```

This will show the log messages per individual test.

```
pot:  750  verbosity set 3
pot:  751  » POTWASM
pot:  751  » node.js detected
pot:  751  » init done
pot:  751  » ready
Asynchronous Happy Path (Promises) Create a KVS - async
pot:  756  » created new in-memory persister
pot:  757  » new slot 1

Asynchronous Happy Path (Promises) Create a KVS, write to it - async
pot:  760  » new slot 2
pot:  763  » put  foo: bar

Asynchronous Happy Path (Promises) Create a KVS, read unset from it - async
pot:  764  » new slot 3
pot:  765  » get  foo: undefined
```

DEEP TESTS

Deep tests are for contributing to POT JS, i.e., improving POT JS itself. They run in different modes, testing in-memory as well as network storage and have a simulation mode that inspects system behavior in the face of delays, timeouts, and cancelation. They run with node.js and in any browser.

For the purpose of POT JS, Selenium or Puppeteer are less helpful as there is no visual / DOM behavior to test. Rather, real resource limits, thread economy, and deadlocks are the focus beyond functional correctness. Browser tests are therefore run in a real browser, not an emulation.

The suite of tests is mostly uniform across all 18 usage modes but some suites are executed only in certain modes: e.g. cancelation can be tested only in 'simulated' mode; long running memory leak tests are run only in 'memory-test' mode.

TYPES OF DEEP TESTS

```
make explain_tests
```

lists the available tests:

TAGS

Deep regression tests can be run in ~2x3x3 combinable deep test modes:

browser/node x in-memory/simulated/network x standard/stress/memory.

The deep test rules are accordingly composed from three tags. For example:

```
% make node_inmem_stress
```

```
Meaning     Tag     |  Description
--------------------+-------------------------------------------------------
browser     web     |  Javascript running in the browser
node.js     node    |  Javascript running in the terminal using node.js
--------------------+-------------------------------------------------------
in-memory   inmem   |  non-persistent, in-memory storage
simulated   sim     |  simulated storage for testing exceptions
network     locnet  |  local Swarm network storage
--------------------+-------------------------------------------------------
standard    test    |  31 test suites of various flavors
recycle ID  quick   |  like *_locnet_test but re-using the batch id
stress      stress  |  4 longer-running suites; mass & concurrent access
memory      memory  |  9 long-running suites testing for memory leaks
```

DEEP TEST RULES

These are the resulting test rules (for other rules use % make help):

```
Rule                |  Description
--------------------+-------------------------------------------------------
test                |  explain test modes and run jest test suite
jest                |  run installation test suite w/o test mode explanation
nodetest            |  explain test modes and run node_inmem_test
webtest             |  explain test modes and run web_inmem_test
--------------------+-------------------------------------------------------
web_inmem_test      |  browser tests with go pot in-memory persisting
web_inmem_stress    |  browser stress tests with go pot in-memory persisting
web_sim_test        |  browser exceptions tests w/out go pot connection
web_locnet_test     |  browser tests with a locally installed Swarm network
web_locnet_quick    |  like web_locnet_test but re-using the last batch id
web_locnet_stress   |  browser stress tests with a locally installed Swarm network
node_inmem_test     |  node.js tests with go pot in-memory persisting
node_inmem_stress   |  node.js stress tests with go pot in-memory persisting
node_inmem_memory   |  node.js memory leak tests with go pot in-memory persisting
node_sim_test       |  node.js exceptions tests w/out go pot connection
node_sim_memory     |  node.js memory leak tests w/out go pot connection
node_locnet_test    |  node.js tests with a locally installed Swarm network
node_locnet_quick   |  like node_locnet_test but re-using the last batch id
node_locnet_stress  |  node.js stress tests with a locally installed Swarm network
node_locnet_memory  |  node.js memory leak tests with a locally installed Swarm network
```

All node_* tests are run on push by github CI workloads, except *_quick and *_memory.
CI runs all those tests for ubuntu-latest, and all non-locnet for MacOS.

memory leak tests run only with node.js as they are only relevant for servers.

Individual memory leak test cases can be picked via names, e.g.:

```
% make node_inmem_memory TEST="gc-response,gc-kvs-detection"
```

```
gc-response             check whether the Javascript GC can be triggered
gc-kvs-detection        check whether the collection of a kvs is detectable
gc-kvs-creation         leak-test sync kvs creation
gc-put-sync            leak-test sync put with counter as key and value
gc-put-async           leak-test async put with negative of counter as key and value
gc-get-async-fix        leak-test async get of value for fix key "K"
gc-put-get-sync         leak-test sync put, get of random keys and values
gc-put-get-async        leak-test async put, get of random keys and values
gc-put-get-async-del    leak-test async put, get, delete of random keys and values
gc-basics-sync          leak-test sync put, get, save, load, get, delete
gc-basics-async         leak-test async put, get, save, load, get, delete
gc-multiop1-async       leak-test 3x async put, get, save, load, get, delete, get
gc-multiop2-async       leak-test 3x async put, get, delete, get
```

The number of iterations can also be adjusted. Below 50,000 is less usefull. E.g.:

```
% make node_sim_memory TEST="gc-kvs-creation" ITER=20000
```

```
ITER works only when TEST is given. Verbosity can be set with VERB.
VERB works only if ITER is given.
```

CONTINUOUS INTEGRATION

Continuous Integration runs these tests on push to github (.github/workflow):

```
inst-tests-macos.yml:           make jest_test
inst-tests-ubuntu.yml:          make jest_test
inmem-tests-macos.yml:          make node_inmem_test
inmem-tests-ubuntu.yml:         make node_inmem_test
inmem-memory-macos.yml:         make node_inmem_memory
inmem-memory-ubuntu.yml:        make node_inmem_memory
sim-tests-macos.yml:            make node_sim_test
sim-tests-ubuntu.yml:           make node_sim_test
sim-memory-macos.yml:           make clean node_sim_memory
sim-memory-ubuntu.yml:          make clean node_sim_memory
stress-tests-macos.yml:         make clean node_inmem_stress
stress-tests-ubuntu.yml:        make node_inmem_stress
stress-tests-ubuntu.yml:        make node_locnet_stress
locnet-memory-put1-ubuntu.yml:  make node_locnet_memory TEST=gc-put-sync
locnet-memory-put2-ubuntu.yml:  make node_locnet_memory TEST=gc-put-async
locnet-memory-get1-ubuntu.yml:  make node_locnet_memory TEST=gc-get-async-fix
locnet-memory-get2-ubuntu.yml:  make node_locnet_memory TEST=gc-put-get-sync
locnet-memory-get3-ubuntu.yml:  make node_locnet_memory TEST=gc-put-get-async
locnet-memory-ops1-ubuntu.yml:  make node_locnet_memory TEST=gc-basics-sync
locnet-memory-ops2-ubuntu.yml:  make node_locnet_memory TEST=gc-basics-async
locnet-memory-ops3-ubuntu.yml:  make node_locnet_memory TEST=gc-multiop1-async ITER=10000(*)
locnet-memory-ops4-ubuntu.yml:  make node_locnet_memory TEST=gc-multiop2-async ITER=30000
locnet-tests-ubuntu.yml:        make node_locnet_test
```

(*) ops3 is too long-running when used with 50,000 iterations and the CI therefore subject to ongoing changes.

CI results are at https://github.com/brainiac-five/potjs/actions.

The 'standard' tests run against the Go implementation of POT, like the use would be in production.

The extended simulation tests, however, focus on POT JS and its WASM-based interfacing to Go POT, add cancellation and time out cases. They do not interface with the Go POT code but emulate it to arrive at clarity about where a potential error originates.

Latency emulation, cancelled promises, and shaky connectivity as well as retention error conditions will be added to emulate the reality of an unreliable network as ultimate storage layer.

Note that the simulation tests (`node_sim_test` and `web_sim_test`) first change the project configuration (`go.mod` + Go directives) and build an executable (`pot.wasm`) that will not work for production. They will maybe briefly appear to work but they sidestep Go POT Use `make unmock`, or `make test` or `make build` to restore the production target, i.e., build the real `pot.wasm`.

The extended simulation tests use a separate, dedicated, in-memory mock storage to separate them from the Go POT implementation. It is not the same as the in-memory storage of the Go POT module that is often referred to and that is used as default when `new()` is called without parameters.

The tests are made to work in the browser and the terminal, using node.js. In all cases (standard, extended simulation, local network, etc.) the tests are invoked by opening the file `test/test.html` (browser-side) or `test/node.js` (server-side). The file that is altered between the test modes is `go.mod`, plus directives to redirect/replace the real Go POT with `mock/`.

The test harness is designed to bridge the language divide and allow for the emulation of exceptions in one degree of separation (caused in Go, caught from JS). The major relevance of the test harness is to help surface any friction between JS's and Go's resource release mechanisms (JS promises and Go channels) to detect possible leaks. Go's channels are not without peril – they can deadlock – and the challenge is aggravated by their position behind the JS API. The relevant code triggering the exceptional conditions resides in `mock.go`. For its purposes, it has to be part of the `main` package rather than package `pot` in `mock/`.

RUNNING A DEEP TEST

```
npx http-server -c1 . &
open "http://127.0.0.1:8080/test/test.html?tag=in-mem"
```

Or use

```
make web_inmem_test
```

It uses `test.js` and `test.css` to display the results from a suite of 160 tests across the available `put` and `get` functions, and more.

DEEP TEST VERBOSITY

To increase verbosity for test logs in the terminal, using node.js, change the setting

```
potVerbosity = 6
```

in line 6 of `test/node.js`.

For browser tests, the logging in the browser console can be adjusted by uncommenting

```
// globalThis.potVerbosity = 6
```

in line 109 of `test/test.html`.

WHAT YOU SHOULD SEE

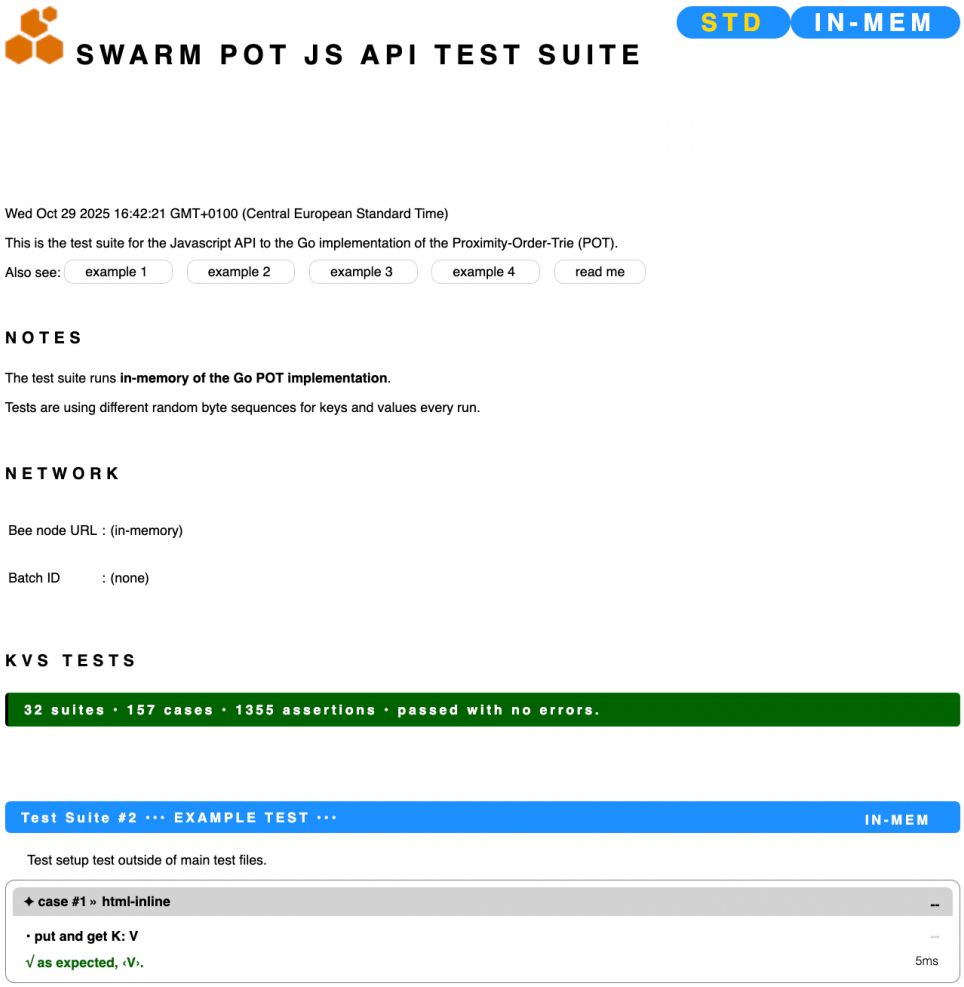
TERMINAL

If you used npx http-server -c1 . & ; open "http://127.0.0.1:8080/test/test.html?tag=in-mem":

```
% npx http-server -c1 . &
Starting up http-server, serving .

http-server version: 14.1.1

http-server settings:
CORS: disabled
Cache: 1 seconds
Connection Timeout: 120 seconds
Directory Listings: visible
AutoIndex: visible
Serve GZIP Files: false
Serve Brotli Files: false
Default File Extension: none

Available on:
  http://127.0.0.1:8080
  http://192.168.178.192:8080
Hit CTRL-C to stop the server

% open "http://127.0.0.1:8080/test/test.html?tag=in-mem"
...  "GET /test/test.html?tag=in-mem" ...
(node:27593) [DEP0066] DeprecationWarning: OutgoingMessage.prototype._headers is
deprecated
(Use `node --trace-deprecation ...` to show where the warning was created)
```

```
...  "GET /test/test.css" ...
...  "GET /lib/wasm_exec.js" ...
...  "GET /test/favicon.ico" ...
...  "GET /test/suites.js" ...
...  "GET /test/test.js" ...
...  "GET /lib/pot.wasm" ...
```

BROWSER



BROWSER CONSOLE

```
pot:  » POTWASM
pot:  » browser detected
pot:  » init done
pot:  » ready
pot:
pot:  ••• Test Suite #2 ••• EXAMPLE TEST •••▶IN-MEM◀
pot:  Test setup test outside of main test files.
pot:
```

```
pot:  ✦ case #1 » html-inline
pot:  --
pot:  • put and get K: V
pot:  › created new in-memory persister
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 1
pot:  › slot ref: 1
pot:  » put  K: V
pot:  › ⟶  4b0000000000000000000000000000…: 0356
pot:  » get  K: V
pot:  › ⟵  4b0000000000000000000000000000…: 0356
pot:  √ as expected, ‹V›.
pot:
pot:  ••• Test Suite #4 ••• SIMPLE GETS AND PUTS OF KVS, SYNCHRONOUS •••▶IN-MEM◀
pot:
pot:  --- b o o l e a n
pot:
pot:  ✦ case #2 » • put K1: false
pot:  suites.js:58
pot:  • new map
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 2
pot:  › slot ref: 2
pot:  √ no error raised.
pot:  • put K1: false
pot:  » put  K1: false
pot:  › ⟶  4b3100000000000000000000000000…: 00
pot:  √ no error raised.
pot:  • getBooleanSync K1
pot:  » get  K1: false
pot:  › ⟵  4b3100000000000000000000000000…: 00
pot:  √ as expected, ‹false›.
pot:
pot:  ✦ case #3 » • put K1: true
pot:  suites.js:74
pot:  • put K1: true
pot:  » put  K1: true
pot:  › ⟶  4b3100000000000000000000000000…: 01
pot:  √ no error raised.
pot:  • getBooleanSync K1
pot:  » get  K1: true
pot:  › ⟵  4b3100000000000000000000000000…: 01
pot:  √ as expected, ‹true›.
wasm_exec.js:22

...

pot:  ✦ case #157 » Oversize Value
pot:  suites.js:4576
pot:  max value size is 100000. Storing the maximal value is part of the stress test
pot:  batch
pot:  • 100000+1 byte sized value should be stopped (async)
pot:  » (re)-using in-memory persister
pot:  » initialize new P.O.T. - slot 121
pot:  › slot ref: 121
pot:  √ no error raised.
pot:  • put 100001 byte buffer raw B
pot:  ### put error: value too large (> 100000 bytes)
pot:  √ expected error: ‹Error: ### put error: value too large (> 100000 bytes)›.
pot:
pot:  ••• Test Suite #32 ••• FAILURES •••▶IN-MEM◀
pot:  Failure tests are available in `simulation` test mode. Use `make web_sim_test`
pot:  or `make node_sim_test`
pot:
pot:  ••• tests cases run: 157 • assertions: 1355  •••
pot:  ▶ no errors ◀
```

DEVELOPING

This chapter preserves insights helpful for contributing to or modifying POT JS itself.

Developing and testing have been done on Mac and Ubuntu, see integration tests in `.github/workflows/` and the previous chapter about tests.

FILES

repo temporarily at https://github/brainiac-five/potjs

```
README.MD              Subset of this manual

potjs.go               Go JS interface
nomock.go              No-op version of test functions
mock.go                Monkey patch mock of Go POT for extended testing
swarm_nodejs.go        Hybrid Go-Javascript persister for node.js on Swarm

mock/                  Replacement of Go POT for extended testing
  pot.go               Mock structures
  go.mod               Go module locations
  pkg/persister        Copy of Go POT persisters to satisfy depencies
    ...

lib/                   Ready-to-use, portable scripts and binaries
  pot.wasm             Go POT implementation compiled to WASM
  wasm_exec.js         Generic Go WASM wrapper,not needed when using pot*.js
  wasm_exec.js.sha384  SHA384 checksum for sub-component verification
  pot-node.js          JS WASM initialization, Go WASM wrapper & fetch code
  pot-web.js           JS WASM initialization and Go WASM wrapper
  pot-web.js.sha384    SHA384 checksum for sub-component verification

doc/                   POT JS documentation
  POT_JS_manual.pdf    Quick start, instructions, tutorial, examples
  POT_JS_manual.odt    Doc source
  POT_JS_function_reference.pdf
  POT_JS_function_reference.odt
  logo.svg

examples/
  README.MD            Overview of examples
  example1.html        Briefest example, in-browser, in-memory storage
  example2.html        Interactive example, input fields, integrity
  example3.html        Example 2 without pot-web.js for more control
  example4.html        Example 1 but using synchronous functions
  example5.html        Example 1 but connected to a local Swarm network
  example6.html        Demonstrating the behavior of the save reference
  example7.js          Example 1 but running in the terminal with node.js
  example8.js          Example 2 but as a single-page node.js web server
  example9.js          Example 8 but connected to a local Swarm network
  example10.js         Demonstration of request canceling
  example11.js         Ways of error handling
  favicon.ico          Swarm logo to suppress browser errors

tutorial/              Files t0.html – t14.js for tutorial of the manual

test/
  README.MD            Explanation of test rules
  jest.async.js        Installation test suite for async functions
  jest.sync.js         Installation test suite for sync functions
  jest.json            Installation test configuration
  test.html            Deep test start for in-browser tests
  node.js              Deep test start for node.js-based tests
  suites.js            Deep test suites
  test.js              Generic deep test harness
  test.css             Generic deep test optics
  favicon.ico          Swarm logo to suppress browser errors

go.mod                 Go module locations
go.sum                 Go module check sums

Makefile               Build scripts

favicon.ico            Swarm logo to suppress browser errors
```

MAKE

The project comes with a comprehensive `Makefile` to support developing.

Below are the build rules for executables and for running tests of POT JS.

Get the most up-to-date list of rules with `make help`.

RULES

Run with `make <rule>`

build	build executables
help	this list of rules
example<n>	start server and run example <n>. n = 1-10
test	explain test rules and modes and run jest installation test
nodetest	explain test rules and modes and run node_inmem_test
webtest	explain test rules and modes and run web_inmem_test
jest	run jest installation test
explain_tests	explain test rules and modes
web_inmem_test	node.js memory leak tests with a locally installed Swarm network
web_inmem_stress	browser stress tests with go pot in-memory persisting
web_sim_test	browser exceptions tests w/out go pot connection
web_locnet_test	browser tests with a locally installed Swarm network
web_locnet_quick	like web_locnet_test but re-using the last batch id
web_locnet_stress	browser stress tests with a locally installed Swarm network
node_inmem_test	node.js tests with go pot in-memory persisting
node_inmem_stress	node.js stress tests with go pot in-memory persisting
node_inmem_memory	node.js memory leak tests with go pot in-memory persisting
node_sim_test	node.js exceptions tests w/out go pot connection
node_sim_memory	node.js memory leak tests w/out go pot connection
node_locnet_test	node.js tests with a locally installed Swarm network
node_locnet_quick	like node_locnet_test but re-using the last batch id
node_locnet_stress	node.js stress tests with a locally installed Swarm network
node_locnet_memory	node.js memory leak tests with a locally installed Swarm network
clean	prepare for building from scratch
distclean	prepare for commit to repository, including build

EXAMPLES

example1	minimal in-browser example as above, in-memory storage
example2	interactive in-browser example with input fields, integrity
example3	like example 2 but without using pot-web.js, plus, delete and save
example4	like example 1 but using synchronous access functions
example5	like example 1 but with local Swarm network storage
example6	like example 5, plus, save and load, WASM path, and verbosity
example7	like example 1 but for node.js
example8	similar to example 2, interactive web app for node.js, in-mem.
example9	identical to example 8, called to use a local Swarm network
example10	simulated demonstration of cancelation
example11	ways of error handling

SUPPORT & DEBUGGING

mock	prepare for extended exception tests
unmock	reset for production or non-extended tests
stop	stop local http server(s) and local Swarm network
http_serve	start local http server serving current directory
http_stop	stop local http server. Ctrl-c does not
locnet_install	install the local test network
locnet_start	start the local test network on docker
locnet_batch	buy stamp batch (free)
locnet_tests	run test suites on local test network
locnet_stop	shut local test network down
locnet_logs	show the entire log of the queen node

DEVELOPER NOTES

The following are rationale, background and learnings on why things were done the way they are. Some circumstances in the interaction between Javascript and Go are unobvious. These notes should save contributors and maintainers a lot of time.

A JS API IMPLEMENTED IN GO

POT JS, while being a Javascript API to Go code, is intentionally and necessarily programmed almost exclusively in Go. It would not work the other way around, i.e., by implementing more – or all – of its functionality in Javascript. But it is also desirable as it is, that it has no core Javascript layer.

There is a Javascript side, `pot-node.js`, and `pot-web.js`. However, only around a hundred lines of code or less of each file form part of POT JS, and they deal almost exclusively with the activation of the WASM mechanism, and don't contribute POT functionality. The exception is the blocking `fetch()` function in `pot-node.js` that is needed by Go POT's synchronous `persister/LoadSaver` model to work with node.js. But even this function is just one page long. The rest of `pot-web.js` and `pot-node.js` represent the generic glue code provided by Go for WASM operation, and a modified Javascript package that enables `fetch()` (see Node.js Swarm Persister, pg. 109, and Synchronous Architecture, pg. 110). However, the WASM executable, written in Go and including the Go POT implementation (proximity-order-trie is the name of that project) constitutes what is required to use POT JS in the browser. `pot-web.js` can even be left out when Go's `wasm_exec.js` is included instead. The actual functionality of the API is all *written* Go. But it is creating, at runtime, Javascript objects.

There is no alternative to this design, which will be explained immediately. But eventually this focus on Go is an advantage. The following explains why it is necessary but also desirable to have POT JS programmed almost exclusively in Go and details why that is the cleaner solution.

WRITING JAVASCRIPT IN GO

POT JS is really mostly Javascript without Javascript code: while POT JS is written in Go, what this Go code creates (cf. file `potjs.go`) are Javascript objects and functions. For all intents and purposes, POT JS, at runtime, *is* mostly Javascript. Just programmed in Go.

The Go code creates Javascript runtime elements that constitute the actual API. Methods like `kvs.put()` are functions written in Go but executed by the Javascript runtime like native Javascript functions.[21] The Go runtime's part is to create them, and also to execute them when they are called.

What is used for the interfacing of Javascript and Go is the Go `syscall/js` package. There is no way to do the opposite using `syscall/js`, or any other tool. I.e., there is no way to create native Go objects from Javascript. `Syscall/js` is a Go package after all, it extends Go, not Javascript. There are translators that create Go code from Javascript code, at compile time. But that is not the same as creating runtime objects for one language in the other at runtime, which is required to create an API as tight and fast as POT JS.

That `syscall/js` works in this direction, from Go to Javascript, is plausible, because the calls are coming *from* Javascript to Go. We want a gate from Javascript to WASM Go code, not the other way around. This is why it makes sense that the connecting elements of the API are created in Go – using `syscall/js` functions – to create Javascript objects and Javascript functions. In a pedantic sense then, while the code is all Go, the API actually *is* fully Javascript. Just created by Go code, not Javascript code. Eventually, Javascript does not 'realize' that it is operating on 'non-native' Javascript objects that are created using Go.

CHANNELS & EXCEPTIONS

Apart from that, there is a reason why at least *some* Go code is going to be required for what POT JS does, and why the mirror image – to write the entire API in Javascript – would not be possible: there are two typical classes of Go objects in play, employed by Go POT (proximity-order-trie), that any Go project would contain: `contexts` and `channels`. Because they cannot be transferred over to Javascript, which does not know the concept of `channels` (`contexts` are special channels), they have to be managed and 'held' on the Go side. This is where `syscall/js`'s explicit incompleteness (see pg. 96) might be hurting. Maybe there could be a way to hand handles to `channels` over to Javascript, and `syscall/js` just hasn't implemented them. This would likely require manual handling of garbage collection on the Go side to prevent the `channels` from being discarded as unused because the Go runtime has no visibility on the Javascript side. The overhead of keeping track of `contexts` and `channels` – either by keeping the objects, or by protecting them from premature garbage collection – necessarily has to be implemented on the Go side.

Therefore, even in its slimmest form, using `syscall/js`, one would always end up having at least some core Go code in the mix. The API could never be all Javascript the way that it is now (almost) all Go.

THE BRIDGE TO WASM

This also trumps another reservation that might be raised: that it would seem much cleaner if one could avoid adding a main package to the Go implementation of POT in order to create a Javascript API, and to instead use Go POT as is, with no additional Go code added for a specific – this – API. Convincing as this view is, it is not feasible. The way `syscall/js` and WASM work it is a given that API-specific Go code has to be added to the Go POT package to use it as WASM executable. It can't work to just wrap Go POT into WASM and be done, all the rest being in Javascript. There has to be

[21] The difference: the Javascript garbage collector does not feel responsible for them, but the Go GC does; and if the WASM executable crashes, they 'disappear.'

some Go code that creates the bridge to the outside world. This cannot be Javascript code. And that Go code has to be part of the WASM executable.

Thus, only compiling Go POT to WASM would not work. One way around it would be to make the WASM act like a server, which would be slow and not the implementation of a Javascript API, but a general interface. And it would require adding code to Go POT just the same. To be clear, Go POT is not changed. It is just made the main component of the API project POT JS.

So, in any event, much of the objects and functions the Go code defines are really Javascript. Thus, much of POT JS is really Javascript, just not Javascript code. Something must be 'the bridge' in and out of WASM and these Go-created Javascript objects are it.

GO PROMISES

Now, if the question were to be asked from a high level, whether having virtually no Javascript code can be a good choice for a Javascript API, the answer is that there is no choice, because the nature of `syscall/js` determines all else. It offers the possibility to 'program Javascript in Go', but not vice-versa. However, even those aspects that would appear to lend themselves to be implemented in a complementary Javascript layer rather than in the Go code – like, e.g., promises – turn out to only work when programmed in Go, instead of in Javascript. The reason is that transacting between Javascript and Go can deadlock the Go code when certain functionality is written in Javascript, instead of in Go. This even includes the creation of promises, i.e. a core functionality of today's Javascript.

Promises being an epitome of Javascript programming, it must seem strange that POT JS creates promises in Go. However, it simply doesn't work in the way that would look more intuitive, i.e., creating a Javascript layer that defines the promises, which's executors then call into the Go WASM functionality synchronously. Below is some background that may help to explain why this is so.

In the end, the answer to why POT JS is mostly programmed in Go is that it is not a choice.

MINIMIZE CROSSINGS

But beyond these hard facts that favor coding POT JS in Go, there are reasons why it can be regarded as the optimal design; the right choice if it was not inescapable at any rate.

From a high-level it is preferrable to have code only, or mostly, on one side – Javascript or Go – and to not cross the language barrier multiple times to keep things robust and avoid unknown unknowns. Keeping things mostly on one side can minimize error sources and overhead that may occur unwittingly when switching between languages, as well as the loss of original state that this entails. `Channels` on the Go side, `exceptions` for Javascript: neither can be 'taken over' to the other side, nor conceptually substituted well. This is why it looks preferrable, independently of the limitations that partially dictate the answer, to keep the bulk of code either on the Go or the Javascript side.

The way `syscall/js` works makes it the Go side.

Keeping things simple – by avoiding back and forth between languages where possible – becomes more relevant for the unsettling disclaimer that comes with `syscall/js`, that it is "experimental and

not for production."[22] There seem to be no complaints out there that it wasn't stable. It may be Google's typical caution, leaving things in beta forever. But it encourages to keep things simple and to avoid more complex constructs where possible. Keeping it all on the Go side then – after some code on the Go side is unavoidable in the first place – appears to be the right choice to increase robustness.

CREATING PROMISES

Eventually, even if all that wasn't convincing enough an argument for pushing everything into Go, it turns out there is a – negative – technical 'killer' argument: as mentioned, it does not work to have asynchronicity being taken care of on the Javascript side. It does not work to have the promise creation – that wraps synchronous calls to Go with asynchronous mechanics – to be coded in Javascript. Specifically, to wrap synchronous calls to Go WASM in Javascript promise executors on the Javascript side. And when using WASM in the browser, a synchronous `fetch` call locks up – as the `syscall/js` documentation warns[23] – It does therefore not work to create the promises in Javascript for the purpose of a Javascript-side layer. Promises have to be implemented in Go. Oddly enough. And POT JS is doing so. Writing the logic that creates Javascript promises on the Go side, works.[24]

DEADLOCKING PROMISES

The `syscall/js` documentation is not clear that this would be required to avoid deadlocks. There is a description on how deadlocks should be avoided by using go routines.[25] But it is not explained why creating the asynchronicity on the Javascript side would be less effective. It can't be because Go would really be running in parallel, because WASM is used with only a single thread. But possibly,

Under certain conditions, this deadlock happens every time. Creating the promise on the Go side requires a go routine to uncouple the promise executor from the synchronous program flow.[26] Just as the `syscall/js` documentation seems to propose. The Javascript asynchronicity inherent in the promise creation, empirically, does not suffice.

This might be because in a 'go executor' call – a Javascript promise executor programmed in Go – the go routine call's mechanics are truly multithreaded, implemented such that if the operating system allows, go routines might run on parallel threads. Maybe this is what is preventing the mechanism from locking up (and/or, the go runtime to 'believe' that it is locking up, when it is done on the Javascript side instead) by virtue of a stronger decoupling that is effected by the go routine call. In that interpretation, the fact that doing the same on the Javascript side – wrapping sync API calls into Javascript promises – always locks up, could be explained by the fact that in Javascript nothing is ever truly parallel, as a basic strategy to keep Javascript programmers safe even while the paradigm is hyper async (except Web Workers, which add true multithreading). Go routines though

---

[22] "This package is EXPERIMENTAL. Its current scope is only to allow tests to run, but not yet to provide a comprehensive API for users. It is exempt from the Go compatibility promise." -- https://pkg.go.dev/syscall/js

[23] "Invoking the wrapped Go function from JavaScript will pause the event loop and spawn a new goroutine. Other wrapped functions which are triggered during a call from Go to JavaScript get executed on the same goroutine. As a consequence, if one wrapped function blocks, JavaScript's event loop is blocked until that function returns. Hence, calling any async JavaScript API, which requires the event loop, like fetch (http.Client), will cause an immediate deadlock." -- https://pkg.go.dev/syscall/js

[24] https://github.com/brainiac-five/potJavascript/blob/86fc8af3ea6b0f148bed1efb34d8dd00190bd1b3/potJavascript.go#L50

[25] "Therefore a blocking function should explicitly start a new goroutine." -- https://pkg.go.dev/syscall/js

[26] https://github.com/brainiac-five/potjs/blob/2bf8d27b3fec175ac6b94cd27805ee3e6aacee47/potjs.go#L1382

are designed to run in parallel if the OS and hardware allow for it. However, this thesis is weakened by the fact that Go in WASM always runs on only one thread. But if the Go deadlock detection's false positives should be the actual problem, this might make no difference.

Regardless, calling from Javascript into Go and then Go calling back to Javascript by making a fetch call seems to only work by virtue of go routine decoupling. Always building such a go routine into a Javascript promise makes for a desirable architecture that has no obvious drawbacks. It even has the priceless advantage that such Go-native promises provide a (the only) way to trigger native Javascript exceptions from Go.

◊

As a wrappers of promises can be regarded as the 'most outside' hull of calls and seeing that even they cannot be implemented on the Javascript side, then little else can. But as noted, for the sake of robustness it looks desirable at any rate to have as much code as possible on one side, and to avoid crossing language lines within one call. The necessary design thus also looks desirable, even if it is surprising that eventually there is only the tiniest Javascript layer in POT JS.

This is why POT JS is implemented almost entirely in Go, and why this is a net positive, too.

NOTES ON DEBUGGING

Being wrapped in WASM to interface with Javascript, the Go executable is opaque. It cannot be debugged as there are no debuggers for WASM, and it becomes unavailable once it has panicked.

Consequently, the development process benefits from being test-driven to unearth unexpected problems. There are 7,000 lines of test code in the `test/` folder. The tests cover a wide array of cases. They can be run against a local Swarm test network, the in-memory storage, or in a special simulation mode that triggers failures.

If unexplainable errors are encountered, increasing the log level to `DEBUG` will be a helpful measure to double check assumptions. Additional logging can be added to the source code using `log(TRACE, "...")`, while setting the log level to `TRACE`.

NOTE ON SYSCALL/JS

The connection between Go and Javascript through WASM is based on Go's `syscall/js` package. This package is officially labeled as experimental by its provider, Google:

> "This package is EXPERIMENTAL. Its current scope is only to allow tests to run, but not yet to provide a comprehensive API for users. It is exempt from the Go compatibility promise."
>
> https://pkg.go.dev/syscall

This reads like a stark warning going beyond Google's typical caution. It would explain the incompleteness of syscall/js, e.g., the lack of an option to directly throw exceptions. What remains unclear, and is worrying, is why this imperfect state has been going on for so long.

The disclaimer makes it just more likely that the browser WASM crashes that report deadlocks are not in fact always deadlocks but could be sometimes or always be a misfiring, overzealous deadlock detection. Especially because in the past the deadlock detection was criticized as too tame in this context, and was then made more aggressive in response.

JAVASCRIPT REWRITE

If Go POT were to be rewritten in Javascript, the central questions would be **a)** whether Javascript can do the bit crunching fast enough, as required to update a POT. Go WASM will be hard to beat. But if this could be shown, also for large tries, or it could be shown that the system never gets CPU-bound but always stays network-bound in real operation, then the implementation of layers should where possible not follow Go POT's blueprint, but be simplified, and on this path **b)** inspected whether it would be possible to replace the persister loop,[27] which is currently implemented by recursion and requires blocking network access, with concurrent, asynchronous writes. While there are strong arguments made against the feasibility of that, it is not inherent to the POT algorithm that it would have to be implemented with blocking network access, but likely rather a price of the flexibility and multi-layered design of Go POT that would not be required for a dedicated Javascript rewrite with a less generalist focus. If it could be done, this would make the operation for node.js much more robust than the current compromise that entails the use of a blocking fetch mechanism that results in a low size limit for values for its overall hacky approach, which could not more clearly go against the grain of node.js. The grain being, simply, that writes are to be asynchronous and node.js really isn't made for a fetch to be synchronous.[28]

In passing, a Javascript rewrite along these lines could also enable a massive performance boost. If the system turns out to be network-bound, which is likely, the parallelization of writing state to storage could be magnitudes more effective than other fine-tuning efforts. It could more than make up for Javascript's lower bit-processing speed.

---

[27] https://github.com/ethersphere/proximity-order-trie/blob/16daa5f7ba9a910ec04dc2db5e72b591840dac7c/pkg/persister/ persister.go#L66-L86

[28] I tried a number of approaches but found that the mechanism that XMLHttpRequest-ssl uses, is the only one that works. It creates a shell process that handles the fetch. Waiting for it to return is the only way, it seems, to be able to block the main thread and neither get a deadlock crash from the Go side, nor end up with async elements that are never executed because the event loop for a function called from Go WASM never starts.

INITIALIZATION

Some details on how the initialization of the WASM executable works. When starting out, the Go WASM executable has to be loaded, both for a HTML5 application or when using POT JS with node.js. This is handled by **pot-web.js**, or **pot-node.js** respectively. These scripts also provide the wait function **pot.ready()**.

potjs-web.js and **potjs-node.js** create the global **pot** object[29] first, before **pot.wasm** is being loaded and they provide in **pot.ready()** a promise that can be used to wait for **pot.wasm** to complete initialization. But **pot**, **pot.ready()**, and the **Kvs** constructor are the only elements of the **pot** object that already exist at that point, no other function can be expected to exist until the promise of **pot.ready()** resolves. Even the **Kvs** constructor does not function yet.

```
    await pot.ready()
    kvs = new pot.Kvs()
    ...
```

cf. examples/example1.html

The core of **pot-web.js** and **pot-node.js** are a few lines of code each that can also be programmed directly in your source code if you are developing for in-browser. **example3.html** shows, how. **pot-web.js** can be omitted then (but **pot-node.js** cannot, because its implementation of **fetch()** is needed).

```
<script src="lib/wasm_exec.js"></script>
<script>
    var kvs
    const go = new Go()
    window.potVerbosity = 3
    WebAssembly.instantiateStreaming(fetch("lib/pot.wasm"), go.importObject)
            .then(async (r)=>{
                    go.run(r.instance)
                    kvs = new pot.Kvs()
            })
    ...
```

part of examples/example3.html

With *this* setup, the global **pot** object is created only *after* the **pot.wasm** executable has loaded. Before that point, the **pot** object and the functions *do not exist* and trying to use them will cause the expectable Javascript exceptions to be thrown. **pot.ready()** will also not have been created, which

[29] A stub, to be precise.

is why onPotInitialized() must be used instead to catch the moment that working with the pot function can begin. onPotInitialized(), if it exists, will be called by pot.wasm as soon as the initialization is done (and the pot object then does exist). It is almost at the same time that pot.ready() resolves.

```
global.onPotInitialized = () => {
        ...
        kvs = new pot.Kvs()
        ...
```

examples/example8.html

The pot.wasm executable loads quickly. Most of the time, it will not strictly be needed to use pot.ready() or onPotInitialized(), as demonstrated by example2.html. In that example, a user has to input data in form fields, before the first operation on a pot object is due. This will hardly ever be fast enough to race the initialization of pot.wasm. The interaction coming from the user will practically always happen only after pot.wasm has long been loaded. It could be a source of error though in the very unlikely case of a network hic-up just at the right moment. But even if during development there is never a problem, the startup sequence should be implemented cleanly to avoid occasional crashes, e.g., for users with a shaky internet connection.

Processing in a node.js program will generally be too fast to ignore the need to wait for pot.wasm to finish loading. You will immediately see errors if you try it without pot.ready() and onPotInitialized(). Note also that with node.js you can't leave pot-node.js out if you want to store to a network because pot-node.js contains a special implementation of a synchronous fetch that is required for the POT algorithm as implemented in Go POT.

TYPES

AUTOMATIC CONVERSION

POT JS automatically casts types to behave in the least surprising way. Here is the rationale for that.

A Javascript developer will make assumptions about how types work. Javascript being weak-typed rather heightens expectations of automation of type handling and on-the-fly casting. To provide the least surprising solution,[30] Javascript types are automatically converted by POT JS to and back from the raw binary storage format that values are stored in with the POT algorithm, to come back as-

[30] *en.wikipedia.org/wiki/Principle_of_least_astonishment – "In user interface design and software design, [1] the **principle of least astonishment** (POLA), also known as principle of least surprise, [a] proposes that a component of a system should behave in a way that most users will expect it to behave, and therefore not astonish or surprise users."*

stored. This automatism can be shirked using the `*Raw*()` functions (e.g., `putRaw()`) that write and read the pure binary payload. `getRaw()` always returns a Javascript `Uint8Array`, no matter what type the value was that was put with `putRaw()`.

The Go POT implementation, per se, stores raw bytes. In Javascript, the equivalent to raw bytes is `Uint8Array`. But using `Uint8Array` values instead of numbers and strings is cumbersome and not idiomatic to Javascript. When thinking of a Javascript key-value store, a question about types does not come to mind first. Of course, values should come back out as thy went in and that would include their type, especially the primitive ones. `Uint8Array`s cannot be used for keys in Javascript,[31] and cannot be compared with each other. They are generally less practical as values than primitives in Javascript, for a lot of use cases. With a tool like POT JS the expectation would be that sending a variable of a certain type to a key-value store should not return later as some raw data that has to be type-cast to be what it once was. Lastly, to oblige the retrieving code to have knowledge about the expected type to correctly interpret the binary data retrieved, in a dynamic Javascript setting can be seen as a conflation of concerns. As JSON showcases, type information tends to be associated with the data.

For PO JS To achieve type-awareness, type encoding has been implemented that can put and get Javascript values without losing their types. It writes a first byte to any typed `put*()` (that is, any put functions that does not have `Raw` in its name) that reflects the Javascript type. Thus, the `get()` and `getSync()` functions can return the correct type for the stored value.

This makes data stored with `put()` (mostly) incompatible with all raw `get` functions. These are `getRaw()`, `getBoolean()`, `getNumber()`, `getString()`, `getRawSync()`, `getBooleanSync()`, `getNumberSync()`, `getStringSync()`.

The assumption is that more advanced applications will prefer full control and use raw getting and setting to store data in their preferred structure, using `Uint8Array`. Programmers new to Swarm POT JS, however, should be welcomed with a more comfortable experience – the automatic type conversion.

While both approaches have merit, the potential for errors is nasty. This is why introducing a flag was considered to force a user to state explicitly, whether raw access should be allowable.

NUMBERS

Numbers are a special case. POT JS stores Javascript numbers to Swarm as their IEEE 754 floating point standard byte code. Direct access to the bits representing numbers guarantees lossless conversion, especially in edge cases.[32]

[31] Because they do not have a natural order in Javascript.

[32] The alternative of storing numbers as strings works well, too, but goes through a much more involved process of conversion that can be assumed to be much more time and memory-consuming. While in the light of this context this does not matter too much, it might also be more prone to errors in edge cases such as the likes of `10/3`, `-0`, and *multiple* variations of `NaN` in IEEE 754.

8-byte float is the universal number type in Javascript. Even integers, other than the special case of `BigInt`s, are internally represented as 64-bit floats. `BigInt`s are unfortunately not covered by POT JS, because `syscall/js` can't handle them. For POT JS, The principally desirable representation of integers by their hexadecimal byte corollary was used as it would have meant special casing the number conversion, i.e., handling integers and floats differently depending on their face value not by any distinction Javascript itself knows or cares about. It would make some Javascript numbers into something they are not (integers), for little tangible advantage other than hypothetical portability of POT storage between clients of different languages.

If cross-system portability of POT data access becomes a goal at some point, then the higher robustness of an integer representation for integers would be convincing – *if* the input could reliably be known to have been meant as an integer in the first place, which in Javascript would remain difficult to ascertain. Instead, for hypothetical integer portability, an additional type marker could be added that identifies integers and allows Javascript to read them but *not write* them unless it is per a special function.

BIG NUMBERS

Javascript `BigInt`s could not be implemented at this point because the Go `syscall/js` package – being officially incomplete (see pg. 96) cannot handle them. It seems likely that `syscall/js` could be extended to make `BigInt` work if desired. There are receipts for it online.

While modding the standard Go package for WASM communication with Javascript opens a can of worms, it seems important enough to eventually allow for `BigInt`s to be stored in Swarm natively.

KEYS

As explained in the Instructions (pg. 9), there are multiple ways that the same key can be expressed. It may be desirable to in the future to guarantee that no two keys ever access the same value.

The current situation exists because there are multiple types allowed for a key, which is idiomatic to Javascript. But different from values, keys are not type-tagged. Javascript `numbers` are stored as their `float` byte representation and zero-padded by POT JS to arrive at the binary 32-byte key that the POT algorithm requires. `"1"`, `1`, and `"0x01"` will thus be different keys but `1`, `1.0`, `0x1` and `01` the same. Meaning, when you store something under the key `"1"`, it will *not* be overwritten by storing something under the key `1`. But when you store something with the key `1`, it *will* be overwritten by storing something under the key `1.0`. Worse, the 8-byte float representation of a number might be matched by a *string*: the string `"A"` matches the bit representation of `131,072` in 64-bit IEEE 754 when both are zero padded to 32 bytes (`0x41` padded with 31 zero bytes).[33] That the keys are padded with zeros presents an almost unescapable challenge to make keys unambiguous.

Changing this type-blindness would require making `Uint8Arrays` the only allowable keys, or to drop `Uint8array` as allowable input for key – which might be more useful – or to implement type-coding for keys like it exists for values. Because even if the 8-byte string/number match would be

[33] Explanation and try-out at https://www.h-schmidt.net/FloatConverter/IEEE754.html

ignored as rare fluke – not a strong choice – any encoding of `number` or `string` keys could also be expressed with a `Uint8Array` in what would by Javascript standards not be regarded as the same value. On the other hand, arguably, being somewhat ad-hoc with automatic type casting is a fit with Javascript style. The existence of the === operator indicates that not everything that looks equal is considered 'strictly equal' in Javascript. The ambiguity of POT JS keys is in the same league.

TESTS

There are multiple kinds of tests. One over-arching distinction is between the *installation tests* that are JEST-based, and the *deep* tests that use a custom harness to run single-source across node.js and browsers.

To accommodate for the dual origination of errors – on the Javascript or Go side – the deep test suite (`test/suites.js`) utilizes the same JS ↔ WASM ↔ Go channel that the API itself uses to trigger, evaluate and log its tests. A number of functions were added to enable black box testing. Most are no ops in production, neutered via Go build directives. Cf. separate doc, '**Function Reference**.'

Part of the deep test suites' capabilities, the 'simulation' tests implement a mock back end (`mock.go`) to be able to test POT JS's reaction to delays, stalls, exceptions and cancelations, and to insulate tests of the JS API from any potential problems with the Go POT implementation itself, which is not yet battle-hardened or extensively tested. This constitutes a third in-memory storage variant, separate from the in-memory storage and the `SingleOrder` mode that the Go POT implementation itself offers and that is used in the 'standard' deep tests and the JEST-based 'installation' tests.

This mock back end – besides being helpful for debugging – emulates fringe conditions that could not otherwise be tested but must be expected when the Go POT package operates connected to a by necessity-unreliable public network. E.g., mock delays are essential to test the basic user cancelation and time out design that has to satisfy both Javascript and Go language idiosyncrasies to prevent resource leaks in Go from dangling sub routines (sic) that were not freed when a request did not succeed. As a special complication, both runtimes – Javascript and Go – have their own garbage collector, which inevitably add opaqueness but, in the instance, have to be made to work together in their individual estimation of what resource can be discarded. In this regard, Go channels are a special challenge, as well as a potential source of deadlocks.[34] This is why the 'simulation' creates an environment that tests the API alone, with harsher conditions.

Regarding stress tests it is unclear whether the existing stress tests put maximal pressure on POT JS and by extension, Go POT, in terms of concurrent operation. Their design might not be right for that and it has not been deeply researched. On the other hand, the multiplexing that protects from write races (`index.go`, `muxProcess()`[35]), as well as the general single-threaded nature of WASM[36] may invisibly be enforcing sequential execution at any rate, preventing certain types of races at any rate.

---

[34] See xxx

[35] https://github.com/ethersphere/proximity-order-trie/blob/4c3949082823e50a8074a7703534ebddcaa66a2a/index.go#L57

[36] See xxx

THE DEEP TEST HARNESS

The deep test harness is custom made to be able to run the same suites in both browser and node.js without changes; to be able to emulate specific kinds of concurrency spanning Javascript and node.js, and failures that can occur in the intersection of the two runtimes; and to seamlessly simulate Go POT to test POT JS alone, to be able to vouch for its correctness, in case of bugs of unclear origin.

The harness consists of `test/test.js` and `test/test.css`. The test suites are in `test/suites.js`. The tests are started via `test/test_node.js` (for node.js) and `test/test.html` (for the browser). They are coordinated via `Makefile` rules that are complex enough that they have to be regarded as part of the test mechanism. `make explain_tests` and `make help` list the different tests, as listed on pg. 90.

The test harness, i.e., the capabilities of `test.js`, are documented by the comments in `test/test.js`. All its features are employed across the ~7,000 lines of tests in test/suites.js and it is straight forward to build new tests analogue to the existing ones.

The general pattern for synchronous function tests is as follows:

```
T.head("...")

T.start("...")

T.log("...")
  ...
T.assertNoError(t0, T, !map)

T.log("...")
  ...
T.assertEqual(t0, T, val, val1)
  ...

T.start("...")
  ...
```

For async tests:

```
T.head("...")

T.start("...")

try {
       T.log("...")
       ...
       T.assertNoError(t0, T, ...)
```

```
        T.log("...")
        ...
        T.assertEqual(t0, T, val, val1)

} catch(err) {
        T.attestUnexpectedError(t0, T, err)
}

T.start("...")

try {
        ...
}
```

ERROR HANDLING

The divergent error handling philosophies of Javascript and Go had to be balanced: a Javascript programmer does not expect errors coming back as return values – like in Go – but to be thrown as exceptions. But Go through `syscall/js` cannot trigger Javascript exceptions through WASM. So the Javascript functions that are really programmed in Go (`potjs.go`) cannot throw errors. They can only `return` a value of type `Error` – at least as JS-native `Error` type – but not `throw` it directly. This may be due to the incompleteness of `syscall/js`; or it may be impossible to implement and even if `syscall/js` was complete one day it could not offer this.

While Javascript `throw` is not available in Go, a middle ground is provided by JS `promises` that enable – when created in Go – by the use of their `reject` function a means to trigger an exception in Javascript. This constitutes an additional reason for why the API functions that return promises are preferrable over the synchronous functions (e.g., `put()` over `putSync()`). This allows for the most seamless transportation of errors from Go to Javascript in the expected way for a JS programmer.

Additionally, calls from Javascript to WASM, the internet finds, should generally be promises as the nature of WASM is not quite like a library without agency but more like a parallel process. (It is not literally a parallel process, as it is executed on the same thread as the JS main thread, but it has to keep running to be available. The Go main function of the WASM executable is put in a never-ending wait with `<-c`).

Synchronous POT JS calls (the POT JS functions with `Sync` in their name) cannot benefit from this way of throwing errors. They return a Javascript `Error` object instead. This does not fit well in the Javascript workflow but there seems to be no better solution. It does not work to create a `promise` on the Go side that in case of an error immediately rejects. This still leads to asynchronicity, literally,

between the error and the arrival of the error message, and "`uncaught (in promise)`"-type errors that arrive a little bit late. E.g., after a test program has finished. Another solution is described in `https://stackoverflow.com/questions/67437284/how-to-throw-js-error-from-go-web-assembly`: the WASM compiler could be extended using a *hint*, and the generic `wasm_exec.js` enhanced to add the possibility to `throw` from Go.

But because this would only be interesting for synchronous functions, which are not assumed central to the overall user experience, the effort and the risk of touching the compiler and `wasm_exec.js` – where until now they are used without modification – seem inappropriate. The less-than-optimal solution will remain for now that synchronous functions return error objects when things go wrong. And that async functions that return promises, and throw errors, should be preferred.

SLOTS

There is an architectural decision that splits the state of a KVS between Javascript and Go. The reason is that some essential Go state associated with a KVS *cannot* be handed over to Javascript, because of its nature: channels and complex Go objects.

The map of `Slot` structures – `SlotMap` (in `potjs.go`) – keeps the part of the state of a KVS on the Go side that cannot be transferred to Javascript. A different, more elegant solution would be preferrable. As things are, and possibly because of the limitations of `syscall/js` (see pg. 96) this might be the best solution. The implementation was a bit of a mine field, but it is sufficiently simple to be robust at this point and is tested literally a hundred million times now.

The key of a `Slot` in the `SlotMap` is a random hash. It is given to the Javascript KVS object as a string. The `Slot` struct is retrieved on every call to `get()`, `put()` etc. accessed through the `this` pointer that is the first parameter that every `js.Func`-wrapped Go function (a Javascript function written in Go) receives when it is called from JavaScript. `this.slotRef` is used to access the right Go `Slot` struct of the KVS in the `SlotMap`.

The most important element in the `Slot` struct is the Go `SwarmKvs` pointer to the struct that is returned from `newSwamKvs()`. It holds, via Go `pot.SwarmKvs`, `pot.Index` that is the struct holding the channels that are required by `muxProcess()`, which is the function that loops through a `select` that effects a write lock, sequencing write accesses to a POT. Every KVS has its own. It kicks in when it is created with `new`.

HYPOTHETICAL ALTERNATE APPROACHES

The slots hold the Go objects and `channels` for which it is not obvious how a safe Javascript handle could be created – one that would not create unsolvable garbage collection issues.

A handle for a Go object could, e.g., be created from its memory address and given to Javascript. But this might have the effect that Go destroys the object if it detects that it has become unreachable by Go's count. Even if it was still needed on the Javascript side. A device to prevent this, Go's `runtime.keepAlive()` is designed to work within only one function.

CLOSURES

A strategically created closure could keep variables alive within it and could prevent the collection. But it would create no less overhead than a straight map of state – like `SlotMap` – that holds the information, and by that in passing prevents its collection.

A yet more involved solution could be tried by enclosing Go variables in a `js.Func` closure that can then be accessed through a wrapping Go function because the variable exists in both the inner and outer closure. But the outer closure itself cannot be attached to the JS object itself and would still have to be held somehow on the Go side – e.g., in a map or array. One would again resort to a map or an array to retain access to the relevant Go objects. This looks no better than storing the state itself in a map or array.

A thinkable alternative to the `SlotMap` would entail the use closures to hold the state that cannot be transferred to Javascript and attach those closures to the Javascript object that is created for a new KVS. This works, relatively elegantly, for the Go context `cancel()` functions of the Javascript promises.[37] They are preserved in a closure that is attached to the promise object. For the state that is now held by `SlotMap`, a Go closure holding it could be wrapped in a `js.Func` – a Javascript function that is created in Go and visible on both sides – and could then be called by calling the wrapping Javascript functions from within Go (sic). State could be accessed from Go when wrapped in this way despite the fact that a `js.Func` must normally be a Javascript object. This is expressed by the `js.Func` type returning the most malleable Go type, `interface{}`. This can be used, even conditionally (i.e., under some circumstances such function could also return a `js.Value`), to return a `channel` out of the function that is then cast from `interface{}` to `chan` by the receiver. This was tried half-way and could work. But in the end it would be overengineered and no more elegant than keeping the bland `SlotMap`.

This is why a state map like `SlotMap` may be the best solution, even while the `cancel()` functions of the promises work without such a central list. It simply keeps Go state on the Go side.

ABANDON CHANNELS

In the face of the affordance to store the channels, it could be a worthwhile investigation to ask if the channels, which serve as a write-locking/sequencing mechanism, are necessary in the first place. Especially given that the entire system is strictly single-threaded and in part *suffering* from this. It is possible that `muxProcess()` – for *this* use case, i.e. with Javascript[38] – is overkill. This discussion ignores that they might still be needed for other uses of Go POT.

[37] https://github.com/brainiac-five/potjs/blob/2bf8d27b3fec175ac6b94cd27805ee3e6aacee47/potjs.go#L1406-L1421

[38] The Go POT implementation might be used in different ways, independently of POT JS.

Possibly, a different locking mechanism than `muxProcess()`'s `select` could be implemented on the Go side as Javascript's single-threaded nature could be taken to represent a free replacement for a multiplexer. Hypothetically, `muxProcess()` was not needed in its current form when interfacing to Go POT from Javascript, neither from the browser nor from node.js, unless Worker Threads were employed that would bring with them true multi-threading. The logic behind the question would be that Javascript is generally single-threaded and thus needs no protection against parallel writing. Javascript does for this very reason not have primitives against it like Go has in its channels. Worker Threads are severely limited to keep true parallelism out of the main program. But for POT JS and Go POT one might ask whether the locking aspect is a duplication of sequencing. Javascript, being single-threaded by nature, does nothing in parallel ever, which `muxProcess()` doubly makes sure can't happen (for writes) – which might as unintended side-effect cause the not quite explained deadlocks of synchronous calls in the browser, which might be false positives of Go's deadlock detection. Perhaps the channels' and `muxProcess()`'s constantly waiting `select` would not be required in the first place, and thus, the channels could possibly be left out altogether or could be replaced by a global semaphore, one per thread and not per KVS, and/or created per-call instead of constantly running in a go routine, so that they do not have to be kept available between calls – and a handle to them in the `SlotMap` – for every KVS object. This could be a modification to make Javascript and Go play together more smoothly.

If this would work, it would leave the second type of `channels` to deal with, the `contexts`. They are presently created per-call so they do not have to be kept around between calls. Though at the cost of losing the possibility of a KVS-wide cancelation option – because a `context` for an entire KVS that all per-call `contexts` derive from would also have to be held somewhere. However, the relevant element of such `contexts` is the `cancel` function and *that* can be kept around in a `js.Func` closure attached to the KVS. It does thus not need the `SlotMap` as connector between Go and Javascript.

There is another relevant object held in `Slots`, a reference to the `persister` associated with a KVS. As this is an object, not a channel, and is not currently keeping a connection alive between calls, it could in its production variant – that connects to a Bee node via `http` – be implemented in a form where its state could be transferred to Javascript, if it needs state at all. Alternatively, the `persister` could be architected to be a native Javascript object, implemented in Javascript code. The bigger obstacle in this case would be the special, state-holding, in-memory version of the `persister` that has to be kept alive between incoming calls from Javascript to not lose its state. But maybe the overall architecture should not be bent to make this less-important, in-memory variant of the `persister` easy to implement. To the degree it is practical for testing, it should rather be the `persister` code itself where the kludge should be residing that makes state-holding between calls possible; instead of burdening the overall architecture for it. To say, to the degree the in-memory persister forces the overall architecture to be suboptimal, it should rather resort to hacks internally, in the persister implementation, instead of holding the architecture back.

But even for such streamlined version of Go POT and POT JS where Go POT is altered and made less broadly useful to work better with POT JS, `muxProcess()` might turn out to *not* be superfluous:

Leaving it out would mean to bank on that there will never be two write calls overlapping each other coming from JS in a way that would trip up Go POT.[39] As is, Javascript's asynchronicity could be enough to have two write promise's executors end up switching back and forth in a way that they look parallel to the Go side. In the end the promises are there to make things look like they happen in parallel and have multiple things pending to speed things up. That's exactly what `muxProcess()` is there to prevent for writes. `muxProcess()` effectively allows multiple writes to pend but prevents them from actually happening at the same time. This sequencing is what `muxProcess()` provides. Trusting Javascript's single-thread would allow for separate jobs to be pushed up to the point where it looks like the writes are both happening and then wait for the result, it does not queue them. This would mean that `muxProcess()` is required, even for a single-thread system.

Finally though, Go compiled to WASM does itself only run on one thread and cannot use its potential to multithread. It is not the case that a speed up through parallelizing and multiplexing could be expected, with parallel writes for different KVS, even while parallel writes for a single KVS is what `muxProcess()` exists to prevent. It is also not in this constellation the case that Go's potential for multi-threading is a useful capacity of Go's that Javascript just can't deal with and that it thus would have some high-level consistency that one had to park the controlling state (the `channels` `muxProcess()` uses for locking) of this Go superpower outside. Go itself can when compiled to WASM not do what `muxProcess()` is designed to prevent it from doing in the wrong moment: it cannot work things in parallel. It would be plausible to expect that `muxProcess()` will at least prevent that two writes race each other by switching back and forth between the two greenlets in an unlucky way. However, to achieve the same protection, it would be enough to just not use go routines in-between to make sure that no two writes could race each other because WASM is executed on a single thread by Javascript, without switching during execution. The conclusion would have to be that `muxProcess()` is unnecessary when Go POT is used in WASM. But this is a special case for Go POT, unlike its potential use in other environments, e.g., when used in Go as a library or package of a bigger system, e.g., included in a Bee client. Only when used with Javascript there is no need to take care of the case that `put()` calls could be coming in on different green threads, much less truly parallelly. So a change or re-implementation of Go POT would target only this case, using Go in WASM, interfacing with Javascript. At any rate, that should have to be weighed against the approach to re-implement the POT algorithm in Javascript and do away with the entire dual-language overhead.

Together with surprises and limitations that surfaced while joining Javascript and Go, it looks difficult to judge whether an attempt at improving this situation would be worth the try.

SLOTS MIRROR A POWER IMBALANCE

In a holistic view, the reason why `Slot`s exist in the first place, is that the sequencer state – the `muxProcess()` channels – would ideally be carried over from Go to Javascript to be held there, which we can't and thus have the `SlotMap` for. Though what's technically only needed, instead, would be to either make sure there is no asynchronicity *started within* Go POT during the handling

[39] Apart from the fact that some future browser optimization might break guarantees that are currently implicit but not spelled out, a browser might do something 'helpful' that ends up using more than one thread to speed Javascript up and by that break the single-thread assumptions. This is not unrealistic because any measures such improvement would require building into the Javascript runtime would be invisible and unavailable when control is given over to WASM. However, that is not on the horizon.

of one `put()`, or, to have a simple semaphore per-KVS on the Javascript side that makes sure that no two write calls are triggered from Javascript at the same time.

In the latter case, there could still be multiple promises for `puts` created and pending at about the same time, but they would simply have to prevented from running their executors to the blocking point of a write at the same time, and so to not rely on asynchronicity alone that would only avoid blocking for Javascript but not a write race on the Go side. Abstractly, this would seem to not change much, because at present, that queuing is happening in `muxProcess()` on the Go side. But it would be a simpler model if that was handled outside of the Go POT core, it would be more robust and possibly apt to avoid any deadlocks of the Go WASM executable that might be but false positives of an overly sharp deadlock detector of the Go runtime (that was historically too lenient).

In light of all the considerations above, the simplistic `SlotMap` looks like the best solution at this point. Its inelegant ways are somewhat explained by how it is an artefact of the overall architecture not being optimal from the Javascript perspective. With its possibility for true parallel execution, Go POT is more powerful than what JS can make good use of. Figuratively speaking, that power needs to be parked somewhere, unused, in the form of the `channels`.

Go POT is meant to have the capability to work truly in parallel. Perhaps, on the one hand, the index updates could with big tries tilt from network-bound into processing-bound. That is, the bulk of the wait-time could at some point be caused by the calculations happening (in Go POT, i.e., in WASM) rather than being caused by the wait for an answer back from the Bee API. On the other hand, how `muxProcess()` is crafted is just the Go way of implementing asynchronous calls to the storage network.

In this view, the construct of the `SlotMap` is an artifact that betrays a friction across the overall implementation – joining Go and Javascript – and stemming necessarily from that Go POT was not made as a Javascript backend, and can do more than Javascript can take from it, namely work in a truly parallel fashion. The `SlotMap` deals in exactly those Go elements that Javascript cannot hold, i.e. the channels that serve something that Javascript was not designed to do: real multi-threading that needs real locks. And as a consistent consequence, Javascript does not have the means to hold the special type of state – the channels – that is native to Go to make true multithreading possible.

NODE.JS SWARM PERSISTER

The topic of network access for WASM executables run by a node.js script is relatively sparsely discussed on the internet and some information found there is misleading. There is a simple answer:

NO NETWORK ACCESS FOR WASM UNDER NODE.JS

Different from a browser, Node.js does not allow any network access for a WASM executable. DNS, TCP, UDP, any fetch access fails for Go running in WASM when executed by node.js. It works in the browser though. This limitation is partially documented but the situation is less than fully clear. In the end, WASM was made for the browser – uniquely, to allow for speed like that of a compiled

language. That WASM could also be used with node.js was probably at most an afterthought. WASM with node.js was implemented to a degree as 'emulation.' Node.js was also not made to host things like a WASM executable. This is visible from the length to which Go's `syscall/js` has to go to make things transparent for the Javascript runtime.

At any rate, while in the browser, network access is extended to the WASM executable 'for free.' The sandbox that restricts the WASM code when running in node.js is stricter. Plausibly, because there is no sandbox around node.js like there is around a *browser* itself, which separates it from the filesystem. Node.js lives on the other side of that divide, with full local access. Presumably that is why it gets no network access.

HYBRID PERSISTER

A new, 'hybrid' `persister`[40] in the mold of the Go Swarm `persister` had to be written for node.js to work with the Swarm network. While the original Go Swarm persister works for POT JS when operating in the browser – i.e., Javascript running in `<script>` tags on a web page – it does not work when used with node.js, i.e. for Javascript projects run locally from the command line.

The new persister, `SwarmNodeJsLoadSaver`, is written in Go and largely identical to the Go `SwarmLoadSaver`. The difference is that it delegates network access to a fetch function implemented in Javascript, `jsFetchSync()`, that uses the code of the node.js XMLHttpRequest emulation package `XMLHttpRequest-ssl` – with a modification, as described below – to realize synchronous network access for node.js, *as* required for the persister architecture, and otherwise near impossible to realize with node.js.

This is an essential step towards a pure Javascript persister, even while other details of it are not clear yet. However, the bolted-on way that `XMLHttpRequest-ssl` is implemented, together with the fact that there seem to be no better alternatives casts a shadow over the idea of a pluggable *pure* Javascript persister. A browser-only version for this concept does not look like a strong decision. But at this point any node.js version, with the idea to have persisters be supplied by users (developers), would always have to find its own creative ways to realize blocking writes and reads, or use the modified version of `XMLHttpRequest-ssl`, which would make the creation not really a custom persister but just a mod of `SwarmNodeJsLoadSaver`.

SYNCHRONOUS ARCHITECTURE

The Go POT implementation (`proximity-order-trie`) is fundamentally a synchronous architecture while POT JS must be asynchronous to work with Javascript, but especially because of node.js.

[40] The persister is a separated-out module that realizes the most basic primitives to store and retrieve for Go POT. It can fundamentally only write data to whatever the target storage is or retrieve it from there. Go POT originally has two persisters, one that stores everything in memory and one that stores everything to a Swarm network.

Go POT waits for writes to succeed before continuing in its `persister`, and has a locking mechanism, `muxProcess()`, specifically to make sure that only one write can proceed at a time. It is unclear whether the algorithm of POT itself demands blocking waits for every write, especially because the network writes appear to be from cache and the immutable, content-addressed entries should be persistable in any order without causing race conditions. While it appears plausible that the progressive calculation of the hashes that tie the trie into one could require a sequential handling of its (updated) nodes, the centrality of the problem caused by the necessity of blocking writes for this, would justify a separation between storage and hash calculation if that is the 'blocker.' And since the current implementation does not have a pick-up strategy if a write at first goes wrong, a failure with concurrent writes, which would at first glance appear to always be more catastrophic, might not be more difficult to repair than a failure in the sequential iteration. However, in practice, it makes sense to assume that this cannot easily be changed and will not be changed because it would in all likeliness be asking for major re-architecting of how Go POT works and how it is compartmentalized into modules.

Conversely though, Node.js quite successfully strives to make synchronous writes all but impossible. Its `fetch` does not have a synchronous option but works only as `promise`. `XMLHttpRequest`, which still works in the browsers and can be used synchronously, has been retired for node.js. There exist popular npm packages to bring it back. `XMLHttpRequest-ssl`[41] employs a blunt hack to evade Javascript's enforced asynchronicity: it spawns a child process that handles the network request asynchronously, then blocks, waiting for the child process to terminate, which it normally does after the response callbacks are completed. The way it is implemented includes creating the child process code as a string, with the payload data hardcoded into it, to start it in a new shell. Headers are singled out as malfunctioning in the docs. Binary payloads and responses did not work with the original `XMLHttpRequest-ssl` library and had to be added and altered.[42] For POT JS this method (that can be improved by using a file to transfer the data to the shell process) currently means a limitation for its value size (to max. ~100,000 byte). That this seems to be the accepted way to attain blocking fetches for node.js just demonstrates how averse node.js is to synchronous network access. But it seems to be the only viable option to create synchronous network requests for node.js.[43]

Of course, node.js' strategy to try to deny programmers blocking fetch calls is sound. Go POT is the outlier demanding them. It would be worth it to find out whether it really needs them.

A modified version of `XMLHttpRequest-ssl` has been included in `lib/pot-node.js`. Requiring `XMLHttpRequest-ssl` as a dependency and modifying it in-situ seemed less sensible as it could be installed in different locations and with limited access. The recent attacks on NPM are an additional factor.

[41] https://www.npmjs.com/package/xmlhttprequest-ssl

[42] The alterations are minuscule, only spanning three lines. But there is no way to make the original package do it without the modifications.

[43] I tried other approaches but they fail because the event loop never starts for JS code called *from* WASM.

GARBAGE COLLECTION

POT JS allows its Javascript users to remain unconcerned about garbage collection. Under the hood, an elaborate dance between the two garbage collectors of Go and Javascript takes place[44] that makes sure that related Go resources are released when Javascript objects go out of scope.

JAVASCRIPT AND GO

The expectation of a Javascript user of POT JS – that no explicit resource releases will be required – is realized by Javascript `finalizers`[45] that trigger a cleanup of Go (sic) resources when the respective Javascript object is collected. For example, when a key-value store is collected by Javascript because it became unreachable, its correlated lock-management and resources on the Go side are let go, too.[46]

The Go team disparages Javascript finalizers in their github comments but this seems to be misinterpretation of their tricky nature that comes across as if they were buggy ('unreliable'), which they are not. That it is unspecified when garbage is collected, and thus, *when* the finalizers trigger, makes it more difficult to reliably test the cross-language collection. This is why the `*_memory` test suites trigger kvs garbage-collection explicitly using `kvs.release()`.

THE TWO GARBAGE COLLECTORS

There is no documentation regarding specifically how `syscall/js` (the API between Go and Javascript) handles garbage collection. On the face of it, things work as they 'ideally should:' Javascript objects created in Go are managed by the Javascript garbage collector as if they were native JS objects; they are slated for deletion when they become unreachable in Javascript – **and** in Go. Which on first glance looks like the perfect arrangement. However, in other words, even when a Javascript object that was created in Go becomes unreachable in Javascript, it is not yet collected by the Javascript GC until it becomes unreachable also on the Go side. Looking at it this way, it becomes less intuitive to handle. Yet worse, the Go `js.Value` that represents a Javascript value *will* be collected on the Go side, and that independently of the fact whether the Javascript object it created still exists! For clarity: the Javascript object is never collected, even if it becomes unreachable from Javascript, as long as the Go variable (`js.Value`) that symbolizes it on the Go side still exists. But the `js.Value` on the Go side *is* collected when it gets unreachable in Go independently of

[44] Garbage collectors discard an object when they detect that it cannot 'be reached' by the program anymore because all handles to it went out of scope – variable names, list entries, uses as a key, closures etc. However, they do this in unpredictable frequency, mostly not the moment the object cannot be reached anymore, and in any order. The clean up callbacks that can be registered are also not necessarily triggered while the object is still accessible but possibly much later. This makes sense to give the garbage collectors all the freedom they need to optimize resource utilization and minimize the time they require for their operation.

[45] https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/FinalizationRegistry

[46] They are deleted from the respective map, which makes them unreachable and subject to collection at an unspecified later time.

whether the actual Javascript object it symbolizes still exists in Javascript. This does make sense but it is not obvious and has consequences. Presumably there is no documentation about this because this design looked intuitive and desirable to the makers of `syscall/js`.

The drawback is that this prevents JS objects from ever being collected if there are still 'internal' references to the object on the Go side. This is described in detail in the issue born of this discussion https://github.com/golang/go/issues/77025:

> Any channels or go routines, especially contexts, that operate under the hood to service a JS object from WASM cannot meaningfully be tied directly into the JS object; there is no representation in JS for them. They need to be held on the Go side with a handle handed over to JS to access them by when needed. This does not necessarily go through the original js.Value, in the sense of the original representation of the JS object in Go.
>
> Go can need active cleanup to release such resources, especially to close channels and stop go routines, the signal for which ideally come from the JS garbage collector at the point when the JS object can be collected – as it is no longer needed on the JS side – to keep resource release automated.
>
> For all I saw, for that signal to ever come – for the JS GC to collect a Go-created JS object in the first place – the js.Value must become unreachable for the Go GC (sic). Because as long as the js.Value that represents the JS object in Go is reachable in Go, the JS object is not garbage collected by the JS GC. This is not documented.
>
> This means that the js.Value must be allowed to become unreachable so the JS GC would ever determine that the JS object that was created through it can be collected. Therefore, [..], the Go resources are always still needed even when the js.Value that symbolizes the JS object they are intended to service is allowed to become unreachable in Go. Otherwise JS GC does not happen. The reachability of the actual JS object on the JS side will determine the actual minimal lifetime of the object. The JS object must thus always not access the Go resources through the initial js.Value, else the JS object is never collected by the JS GC. But that is not how such access works in the first place. Access would come from the JS side for its purposes and the JS object does not know its js.Value or go through it on the Go side. It might re-appear on the Go side as new js.Value representing the same JS object; as this (first parameter, of type js.Value of the JS function signatures seen from Go); or it might be retrieved using Get(<name>). The original js.Value – in the sense of the Go value of type js.Value representing the JS object – is not needed. For GC to work it must be discarded.
>
> It is my point that a runtime.AddCleanup() added to this js.Value that must be discarded, does not work, because when the js.Value becomes unreachable in Go, it is collected even while the JS object it stands for is not (but as stated, if the js.Value is not allowed to become unreachable in Go, the actual JS object is never collected by the JS GC). The Cleanup therefore fires too early; reliably so because the JS object on the JS side is not collected before the js.Value is on the Go side. It is not possible, therefore, it would seem, to tie the js.Value to the actual JS object in a way so that the js.Value is collected when the real JS object is collected, and could at that point clean up.
>
> That's why the one way that works – for all I could find – is to use the JS-side Cleanup mechanism instead, and of the two options, the older FinalizationRegistry. The sequence becomes 1) letting the js.Value become unreachable on the Go side, 2) JS GC triggers finalization once the actual JS object becomes unreachable on the JS side, which 3) calls a Go-created finalization closure to release the resources on the Go side. Because (again) keeping the js.Value blocks any garbage collection by both runtimes, and using AddCleanup() on the js.Value triggers before the JS object is unreachable and could incapacitate it.

```
pot: ------------------------------------------------------------------------------
pot: ✦ case #135 » Crossover Concurrent Putting, Getting, Saving, Loading, Switching Maps
pot: ------------------------------------------------------------------------------
pot: Crossover concurrent putting, getting, saving, loading, switching maps across
pot: threads, with promise
pot:
pot: 1: • new map
pot: » (re)-using in-memory persister
pot: » initialize new P.O.T. - slot 85
pot:
pot: 2: • second new map
pot: » (re)-using in-memory persister
pot: » initialize new P.O.T. - slot 86
pot:
pot: • waiting for completion
pot: » concurrent threads: 2
pot: 1: √ no error raised.                                                    3 ms
pot:
pot: 1: • putK1: V1
pot: » put  K1: V1
pot: 2: √ no error raised.                                                    1 ms
pot:
pot: 2: • get K1 from second map
pot: » get  K1: undefined
pot: 1: √ no error raised.                                                  « 1 ms
pot:
pot: 1: • get K1
pot: » get  K1: V1
pot: 2: √ as expected, ‹undefined›.                                           1 ms
pot:
pot: 2: • put K1: V1 to second map
pot: » put  K1: V1
pot: ⊗ KVS on slot 84 cleaning up
pot: ⊗ KVS on slot 83 cleaning up
pot: ⊗ KVS on slot 82 cleaning up
pot: ⊗ KVS on slot 86 cleaning up
pot: ⊗ KVS on slot 85 cleaning up
pot: 1: √ as expected, ‹V1›.                                                  4 ms
```

Both Javascript and Go have rudimentary mechanisms to free resources manually, designed to, e.g., release file handles of files that might otherwise stay open when the controlling object is collected because it fell out of reach, but had not yet closed its files properly.

The POT JS `Slots` have to be 'released' on the Go side, once they are not needed any more. This is triggered from the Javascript side, when a KVS object goes out of reach there. Or, it can be triggered explicitly using `kvs.release()`. This can make the state and methods of the kvs go away *before they became unreachable for JS code.*

While the Go garbage collector can be triggered manually using `pot.gc()`, this is not the case with Javascript, where only a torrent of lots of expensive memory allocations can trigger a GC run – indirectly and unreliably (cf. `test/suites.js gc-kvs-detect`.)

There are scholarly articles about garbage collection and determinism of execution in Go. And the source code of `syscall/js` (the Go package that allows to interface with Javascript) reveals that Go is controlling the Javascript values' collection 'manually.' They get a `finalizer` set,[47] which calls into WASM. Digging deeper, one runs into explicit warnings like[48]

> "Don't try to use this directly."

It is this bit that may have contributed to the bias of official github contributors regarding Javascript's `finalizers`.

RELEASING FUNCTIONS

Additionally to releasing objects, Javascript functions created in Go must also be *released* manually to free their memory.

POT JS creates such functions for the global `pot` object that are created once and are never released. The functions that are continuously dynamically created are the promise executors and `cancel()` methods of promises returned by `get()`, `put()` etc. The `cancel()` function is a closure over the Go channels of type `ctxContext` that serve to cancel the promise. This is why it has to be created separately for each promise, as it encapsulates a different `ctxContext` every time. It is currently released when the promise resolves but that is not yet ideal as it forces a user who wishes to use `cancel()` to first test if the method still exists, which is not idiomatic to Javascript. But the *optional chaining* syntax (`?.`) should allow for race-free test-and-execution in one.

[47] https://cs.opensource.google/go/go/+/refs/tags/go1.25.3:src/syscall/js/js.go

[48] https://pkg.go.dev/golang.org/x/mobile/bind/seq#pkg-functions

ROADMAP

Possible future improvements:

- NPM PACKAGE
- JAVASCRIPT MODULES
- HIGHER LIMIT OF NODE.JS VALUE SIZE
- HIGHER COVERAGE OF PARAMETER TESTS
- PROOF RETRIEVAL FUNCTION
- TREE WALK FUNCTIONS
- JS-NATIVE PERSISTERS
- TYPESCRIPT VERSION
- ERROR MESSAGE CLEAN UP
- WASM RESTART AFTER CRASH